

Programming, Data Structures & Algorithms — ISRO CSE Revision Notes

Concise notes + relevant exam questions, compressed from the KnowledgeGate PDSA module.

1. C Fundamentals — Data Types & Operators

1.1 Character Set, Identifiers & Basic Structure

- **C character set:** letters, digits, special symbols. **Identifier rule:** 1–31 chars, starts with a letter or underscore, only letters/digits/underscore allowed, no keyword can be reused as an identifier.
- **Constant** = value that doesn't change; **Variable** = named memory location whose value can change. Both declared with a type before use.
- Every C statement ends with `;`. C is free-form (no fixed column rules). Comments `/* ... */` — **TRAP: comments cannot be nested** in standard C.
- `main()` is the mandatory single entry point of every C program; forms: `main()`, `main(int argc, char *argv[])` (`argc`=argument count, `argv`=argument vector), `main(int argc, char *argv[], char *env[])`.
- **Uninitialized local variable** → **garbage value** (leftover memory content, NOT assigned by the compiler). **TRAP:** in Java, uninitialized class-level fields default to 0 — do not assume the same "safe default" behavior in C.
- `extern` declares (doesn't define) a variable — no memory allocated until some translation unit actually defines/uses it.

1.2 Data Types & Sizes (typical 32/64-bit system assumption used by GATE/ISRO)

Type	Typical size	Notes
char	1 byte	signed range -128..127, unsigned 0..255
int	4 bytes	signed range $\approx -2.1 \times 10^9$ to 2.1×10^9
float	4 bytes	~6-7 significant digits
double	8 bytes	~15-16 significant digits

- `short` / `long` modify size (short `int` $\leq$ `int` $\leq$ long `int`, sizes are implementation-defined but this ordering always holds).
- **Real (floating) constants:** fractional form (`426.0`) or exponential form (`3.2e-5` , mantissa e exponent).
- **Character constant:** single char in single quotes (`'A'`), stored internally as its **ASCII value** (e.g., `'A'`=65, `'a'`=97) — arithmetic on chars operates on these ASCII codes.

1.3 Integer Overflow / Wraparound

Unsigned type wraps modulo 2^n : `unsigned char 255 + 1 = 0`
 Signed overflow is UNDEFINED BEHAVIOR in the C standard, but GATE/exam questions conventionally assume 2's-complement wraparound:
`signed char 127 + 1 = -128 (assumed)`

- **TRAP:** Signed integer overflow is technically undefined behavior (compiler could do anything) — but exam questions expect the 2's-complement wraparound answer unless stated otherwise.

1.4 Type Conversion

`int op int` → `int`
`real op real` → `real`
`int op real` → `int` PROMOTED to real first, result is real
 assignment: RHS result is converted (promoted/demoted) to match LHS variable's type

- **Integer division truncates toward zero; float division keeps the fraction:** `5/2` → `2` (int), but `5.0/2` or `5/2.0` → `2.5`.
- **TRAP:** `k = 9/2` gives `4` (int truncation) even if `k` will later be printed as a float context — the truncation happens on the RHS **before** any assignment-based promotion.
- **Modulus %:** only valid on integers (not float). Result's sign matches the **dividend (numerator)**: `-5 % 2 = -1`, `5 % -2 = -1`.
- No exponentiation operator in C — `a ** b` and `a ^ b` are both invalid for "power" (`^` is bitwise XOR!) — must use `pow()` from `<math.h>`.

1.5 Operators

- **Unary:** `++` / `--` (pre/post), unary `+` / `-`, logical `!`, bitwise `~`, `sizeof`, address-of `&`, dereference `*`, type cast — all **right-to-left** associativity.
- **sizeof:** compile-time only operator; any expression inside it is **NOT executed** at runtime (its side effects never happen).

```
int a = 5;
int x = sizeof(a = 10); // a=10 assignment inside sizeof is NOT executed
printf("%d", a);        // prints 5 (unchanged)
printf("%d", x);        // prints 4 (size of int, unrelated to the expression's value)
```

- **Pre vs Post increment:**

Feature	<code>++x</code> (pre)	<code>x++</code> (post)
When incremented	Before use in the expression	After use in the expression
Value used in expression	New (incremented) value	Original (old) value

- **TRAP:** Expressions like `i = i++ + ++i` invoke **undefined behavior** in C (multiple unsequenced modifications to `i`) — different compilers may give different answers; be aware such questions technically have no single "correct" C answer even though exams often expect one based on common-compiler behavior.

- **Bitwise:** `&`, `|`, `^` (XOR), `~` (one's complement, `~5 = -6` in 2's complement), `<<` (left shift = multiply by 2^n), `>>` (right shift = divide by 2^n for unsigned/logical shift; for signed, commonly sign-extends — implementation-defined but usually assumed in exams).
- **Logical (`&&`, `||`, `!`) — SHORT-CIRCUIT evaluation:**

`&&` stops evaluating as soon as ANY operand is false (0)

`||` stops evaluating as soon as ANY operand is true (nonzero)

- **TRAP:** In `a() && b()`, if `a()` returns 0, `b()` is **NEVER CALLED** — this matters enormously when operands have side effects (e.g., `++x`), since a short-circuited operand's side effect never happens.

1.6 Operator Precedence (high → low, practical GATE-relevant ordering)

1. Postfix ++/--	(Left-to-Right)
2. Unary: ++/-- (pre), + - (unary), ! ~, sizeof, (type)cast, & *	(Right-to-Left)
3. Multiplicative: * / %	(L-to-R)
4. Additive: + -	(L-to-R)
5. Shift: << >>	(L-to-R)
6. Relational: < <= > >=	(L-to-R)
7. Equality: == !=	(L-to-R)
8. Bitwise AND: &	(L-to-R)
9. Bitwise XOR: ^	(L-to-R)
10. Bitwise OR:	(L-to-R)
11. Logical AND: &&	(L-to-R)
12. Logical OR:	(L-to-R)
13. Ternary: ? :	(Right-to-Left)
14. Assignment: = += -= *= /= etc.	(Right-to-Left)
15. Comma: ,	(Right-to-Left)

- **TRAP:** Ternary `? :` and assignment are **right-to-left** — a nested ternary `a ? b : c ? d : e` groups as `a ? b : (c ? d : e)`, NOT `(a ? b : c) ? d : e`.

Questions

Q1. [Numerical] What is the output?

```
int a = 2, b;
b = a && !a;
printf("%d", b);
```

A. 0 B. 1 C. 2 D. Compile error

Answer: A. Why: `!a = !2 = 0` (a is nonzero so its logical negation is 0); `2 && 0 = 0` (short-circuits false since second operand is 0).

Q2. [Trap] What is the output?

```
int a = -1, b = 2, c;
c = ++a && ++b;
printf("%d%d%d", a, b, c);
```

A. 020 B. 121 C. 023 D. 122

Answer: A. Why: `++a` makes `a=0` (falsy) — `&&` short-circuits, so `++b` is NEVER evaluated, `b` stays 2; `c = 0`. Output: `a=0,b=2,c=0` → "020".

Q3. [Trap] What is the output?

```
int a = 0, b = 2, c;
c = ++a || ++b;
printf("%d%d%d", a, b, c);
```

A. 121 B. 021 C. 122 D. 020

Answer: A. Why: `++a` makes `a=1` (truthy) — `||` short-circuits true, so `++b` is NEVER evaluated, `b` stays 2; `c=1`. Output: "121".

Q4. [Numerical] What is the output?

```
int a = 1, b = 1;
int c = a || --b;
int d = a-- && --b;
printf("a=%d, b=%d, c=%d, d=%d", a, b, c, d);
```

A. `a=0, b=1, c=1, d=0` B. `a=0, b=0, c=1, d=0` C. `a=1, b=1, c=1, d=1` D. `a=0, b=0, c=0, d=0`

Answer: B. Why: `a || --b` : `a=1` (truthy) → short-circuits, `--b` never runs, `c=1`, `b` still 1. `a-- && --b` : `a--` evaluates current `a=1` (truthy) THEN decrements `a` to 0; since left is truthy, `--b` runs: `b` becomes 0 (falsy); `d = 1 && 0 = 0`. Final: `a=0, b=0, c=1, d=0`.

Q5. [Numerical] What is the output?

```
int x = 10, y = 5, z = 2;
int result = x > y ? y > z ? 1 : z > x ? 2 : y > z ? 4 : z > x ? 5 : 6;
printf("%d", result);
```

A. 1 B. 2 C. 4 D. 6

Answer: A. Why: Ternary is right-associative: `x>y` is true(`10>5`) → evaluate `y>z ? 1 : (...)`; `y>z` is true(`5>2`) → result = 1. The rest is never evaluated.

Q6. What is the value of `k` after `k = 9 / 2`; where `k` is declared `int` ?

A. 4.5 B. 4 C. 5 D. 4.0

Answer: B. Why: Both operands are `int`, so integer division truncates toward zero: `9/2 = 4` (the fractional part is simply discarded, not rounded).

Q7. What is `-5 % 2` in C?

A. 1 B. -1 C. -3 D. 3

Answer: B. Why: The sign of the result of `%` always matches the sign of the dividend (numerator): $-5 \% 2 = -1$.

Q8. [Trap] Which statement about `sizeof(a = 10)` (where `a` is a previously-declared int currently holding 5) is correct?

A. It executes the assignment, so `a` becomes 10 B. It does NOT execute the assignment — `a` remains unchanged; the expression only yields the byte-size of its type C. It causes a compile error D. It always returns 0

Answer: B. Why: `sizeof` operates at compile time and never executes its operand expression, so any embedded side effect (like an assignment) never actually happens at runtime.

Q9. [Numerical] What does `~5` evaluate to (assuming 2's complement, 32-bit int)?

A. -5 B. -6 C. 6 D. 10

Answer: B. Why: One's complement flips all bits; for 2's-complement representation this equals `-(x+1)`, so `~5 = -6`.

Q10. In the expression `a() && b() || c()`, if `a()` returns a nonzero value, which functions are guaranteed to execute?

A. Only a() B. a() and b() only C. a(), b(), and c() D. Only b() and c()

Answer: B. Why: `&&` has higher precedence than `||`, so this groups as `(a() && b()) || c()`. Since `a()` is nonzero (truthy), `&&` proceeds to evaluate `b()` too. If `a() && b()` is truthy overall, `||` then short-circuits and `c()` is skipped — but `c()`'s execution depends on `b()`'s result, whereas `a()` and `b()` are always both called whenever `a()` is truthy.

Q11. [Numerical] Given `char x = 127;` (signed char, 2's complement assumed), what is the value of `x + 1`?

A. 128 B. -128 C. -1 D. 0

Answer: B. Why: Signed char range is -128..127; 127 is the max value, so incrementing wraps around (under the conventional 2's-complement assumption exam questions use) to -128.

Q12. [Trap] Which of the following correctly computes "3 raised to the power of 2" in standard C?

A. `3 ** 2` B. `3 ^ 2` C. `pow(3, 2)` using `<math.h>` D. Both A and B are valid

Answer: C. Why: C has no built-in exponentiation operator; `**` is not a valid C operator and `^` is bitwise XOR (not power) — you must use the `pow()` library function.

2. C Control Flow — Conditionals, Loops & Switch

2.1 if / if-else / Nested if-else

- `if (expr) stmt; — expr` can be ANY valid expression (arithmetic, assignment, even a bare negative number) — C treats **zero as false, any non-zero as true**. So `if(-5)`, `if(a=10)` (an assignment, which evaluates to the assigned value) are all legal and TRUE.
- **TRAP:** A stray semicolon after the condition — `if (i==5); printf("hi");` — makes the `;` an empty statement satisfying the `if`, so `printf` runs **unconditionally**, every time, regardless of the condition. The compiler gives no error; only the logic is broken.
- Multiple statements under one `if / else` MUST be wrapped in `{ }`; without braces, only the single next statement belongs to the `if`.
- **Dangling else:** an `else` always binds to the **nearest unmatched preceding if**, regardless of indentation:

```
if (a > 0)
    if (b > 0)
        printf("both positive");
else
    printf("a not positive"); // actually binds to the INNER if (b>0), NOT the outer one!
```

TRAP: Indentation is just visual — the compiler binds `else` to the closest open `if` lexically, so misleading indentation can hide a logic bug.

2.2 Conditional (Ternary) Operator

```
result = (condition) ? expr_if_true : expr_if_false;
```

- Right-to-left associative when chained: `a ? b : c ? d : e` groups as `a ? b : (c ? d : e)`.
- Equivalent to if-else but is itself an *expression* (has a value), so it can be used inside larger expressions/assignments where if-else cannot.

2.3 Loops: while, do-while, for

Loop	Condition checked	Min. executions of body
while	BEFORE body	0 (may never run)
do-while	AFTER body	1 (always runs at least once)
for	BEFORE body (like while)	0

TRAP: `do-while` is the ONLY loop guaranteed to execute its body at least once — this is the single most tested while-vs-do-while fact.

- `for (init; condition; increment)` — **all three parts are optional**. `for(;;)` is a valid infinite loop. Multiple statements are allowed in init/increment via the **comma operator** (`for(i=1,j=2; j<=10; j++, i++)`), but only ONE expression (possibly combined with `&&` / `||`) is allowed in the condition slot.
- **"Odd loop":** a loop whose exact repeat count isn't known in advance (e.g., "keep asking until user enters 'n'") — typically written as `do-while(condition)` for readability, but can be written with `for(;cond;)` or `while(cond)` identically.
- A loop counter need not be `int` — it can be `float` / `char` too, BUT: an integral counter that overflows its type's range **wraps around** (Ch.1 §1's cyclic nature) — e.g. `int i=1; while(i<=32767) {...i=i+1;}` on a 16-bit-int system never terminates: after 32767, `i+1` wraps to -32768, which still satisfies `i<=32767`, looping forever.

2.4 break vs continue

- **break:** exits the **nearest enclosing** loop or `switch` immediately — control jumps to the first statement AFTER that loop/switch.
- **continue:** skips the rest of the current iteration's body and jumps to the loop's **update/re-test step** (increments `for`'s counter, then re-checks condition; re-checks `while`'s condition directly).
- **TRAP:** `break` inside a `switch` that itself sits inside a loop only exits the **switch**, not the loop — a very common nested-control trap.
- **TRAP:** `continue` inside a loop does NOT restart the whole loop from scratch — it still runs the `for` loop's increment step before re-testing the condition.

2.5 switch Statement

```
switch (integer_or_char_expression) {
    case CONST1: stmt; break;
    case CONST2: stmt; break;
    default: stmt;
}
```

- Works only on **integral types** (`int/char` and expressions evaluating to them) — **NOT float, NOT string**, in standard C.
- Case labels must be **constant expressions** (e.g. `case 3+7:` is fine; `case a+b:` with variables is a compile error) and each case value must be **unique**.
- **Fall-through:** without a `break`, execution continues into the NEXT case (and the one after, etc.) regardless of whether its value matches — this is the #1 most-tested switch fact.
- `default` can appear **anywhere** in the switch (not necessarily last); if no case matches and there's no `default`, control simply falls through the entire switch with no action.
- Multiple case labels can share one action by stacking labels with no statement between them (e.g. `case 'a': case 'A': printf(...); break;`).
- Any statement written before the first `case` label inside the switch body is **unreachable** (the compiler won't error, but it never executes).
- **switch vs if-else ladder:** switch compiles to a **jump table** (O(1)-ish dispatch) — faster than an if-else ladder when there are MANY cases; but for very few (~2) simple conditions, an if-else ladder is actually faster since the jump-table lookup has its own overhead. switch also cannot test floats or range/complex conditions (only exact-match on an integral constant) — if-else has no such restriction.

2.6 goto

- `goto label;` jumps unconditionally to a `label:` anywhere in the same function.

- Generally discouraged (hurts readability/maintainability), but the one broadly-accepted legitimate use is breaking out of **deeply nested loops** in one step (since `break` only exits one level).

Questions

Q1. [Trap] What does the following print?

```
int i;
for (i = 5; i <= 10; i++);
printf("%d", i);
```

A. 5 6 7 8 9 10 B. Prints nothing C. 11 (once) D. Infinite loop

Answer: C. **Why:** The semicolon right after the `for(...)` header makes the loop body an empty statement; the loop just runs to completion (i becomes 11) with no output, and THEN the un-braced `printf` (not part of the loop) executes exactly once, printing `11`.

Q2. [Trap] What is the output?

```
int i = 1;
while (i <= 3) {
    printf("%d ", i);
}
```

A. 1 2 3 B. Infinite loop printing "1 1 1 1..." forever C. Prints nothing D. Compile error

Answer: B. **Why:** `i` is never incremented inside the loop body, so the condition `i <= 3` never becomes false — `i` stays 1 forever, printing "1 " endlessly.

Q3. What does this print?

```
int i = 0;
do {
    printf("%d ", i);
    i++;
} while (i < 0);
```

A. Nothing (condition is false initially) B. 0 C. Infinite loop D. Compile error

Answer: B. **Why:** `do-while` always executes the body at least once BEFORE checking the condition — so `0` is printed, `i` becomes 1, then the false condition `1 < 0` stops the loop.

Q4. [Trap] Output of:

```
int i;
for (i = 1; i <= 3; i++) {
    if (i == 2) continue;
    printf("%d ", i);
}
```

A. 1 2 3 B. 1 3 C. 1 D. Infinite loop

Answer: B. **Why:** When `i==2`, `continue` skips the `printf` for that iteration only and jumps straight to `i++` /re-test; `i=1` and `i=3` both print normally.

Q5. [Numerical/Trap — GATE 2015] Consider:

```
int i, j, k = 0;
j = 2 * 3 / 4 + 2.0 / 5 + 8 / 5;
k -= --j;
for (i = 0; i < 5; i++) {
    switch (i + k) {
        case 1:
        case 2: printf("%d", i + k);
        case 3: printf("%d", i + k);
        default: printf("%d", i + k);
    }
}
```

How many times does a `printf` execute in total?

A. 8 B. 9 C. 10 D. 11

Answer: C. **Why:** $2*3/4 = 1$ (int div), $2.0/5 = 0.4$ (float), $8/5 = 1$ (int div) $\rightarrow j = 1 + 0.4 + 1 = 2.4$, truncated to **int j=2**. `--j` pre-decrements to 1, so `k = 0 - 1 = -1`. Now `i+k` for `i=0..4` is `-1,0,1,2,3`. `i=0` $\rightarrow$ no case matches $\rightarrow$ only `default` runs (1 `printf` each = 2). `i=2` (`i+k=1`) $\rightarrow$ falls through `case1` $\rightarrow$ `case2(print)` $\rightarrow$ `case3(print)` $\rightarrow$ `default(print)` = 3 `printfs`. `i=3` (`i+k=2`) $\rightarrow$ `case2(print)` $\rightarrow$ `case3(print)` $\rightarrow$ `default(print)` = 3 `printfs`. `i=4` (`i+k=3`) $\rightarrow$ `case3(print)` $\rightarrow$ `default(print)` = 2 `printfs`. Total = $1+1+3+3+2 = 10$.

Q6. [Trap — GATE 2012] What is the output?

```
char inchar = 'A';
switch (inchar) {
    case 'A': printf("choice A \n");
    case 'B': printf("choice B ");
    case 'C':
    case 'D':
    case 'E': default: printf("No Choice");
}
```

A. No choice B. Choice A C. Choice A Choice B No choice D. Program gives no output (erroneous)

Answer: C. **Why:** No `break` anywhere — matching `case 'A'` falls through `case 'B'` (prints "choice B "), then through the empty `case 'C' / 'D' / 'E'` labels into `default` (prints "No Choice"). All three `printfs` execute in sequence.

Q7. [Trap] What does `break` do when placed inside a `switch` that itself is inside a `for` loop?

A. Exits the for loop entirely B. Exits only the switch statement; the for loop continues to its next iteration C. Exits both the switch and the loop D. Causes a compile error

Answer: B. **Why:** `break` only exits the *nearest* enclosing loop or switch — here that's the switch, not the outer for loop.

Q8. What is printed?

```
int i, j;
for (i = 1, j = 2; i <= 2 && j <= 3; i++, j++)
    printf("%d-%d ", i, j);
```

A. 1-2 B. 1-2 2-3 C. 1-2 2-3 3-4 D. Compile error (comma not allowed in for)

Answer: B. Why: Comma operator allows multiple init/increment statements; loop runs while BOTH `i<=2` and `j<=3` hold: $(i=1, j=2) \rightarrow \text{true} \rightarrow \text{print}$; $(i=2, j=3) \rightarrow \text{true} \rightarrow \text{print}$; $(i=3, j=4) \rightarrow i<=2$ fails $\rightarrow$ stop. Prints "1-2 2-3".

Q9. [Trap] Dangling-else: what does this print for `a=5, b=-1` ?

```
if (a > 0)
    if (b > 0)
        printf("both positive");
    else
        printf("a positive, b not");
```

A. Nothing is printed B. "both positive" C. "a positive, b not" D. Compile error

Answer: C. Why: The `else` binds to the nearest unmatched `if` — the INNER `if(b>0)`, not the outer one. Since `a>0` is true, we enter the inner if; since `b>0` is false, its attached `else` fires, printing "a positive, b not".

Q10. A switch statement is generally FASTER than an equivalent if-else ladder mainly because:

A. switch uses less memory always B. The compiler builds a jump table, giving near-constant-time dispatch instead of sequentially evaluating each condition C. switch does not require a break statement D. if-else cannot use integer expressions

Answer: B. Why: A jump table lets execution go directly to the matching case's code; an if-else ladder must evaluate conditions one by one until a match — slower especially with many cases (though for very few simple conditions if-else can actually be faster due to jump-table overhead).

Q11. Which case values are ILLEGAL in a C switch statement?

A. `case 5:` B. `case 'A':` C. `case a+b:` (where a, b are variables) D. `case 3+7:`

Answer: C. Why: Case labels must be constant expressions evaluable at compile time; an expression involving variables is not allowed, while a constant arithmetic expression like `3+7` is fine.

Q12. What does `continue` do inside a `for` loop, precisely?

A. Immediately exits the loop B. Skips remaining body statements for this iteration, then still executes the increment step before re-testing the condition C. Restarts the loop from the initialization step D. Has no effect in a `for` loop

Answer: B. Why: Unlike in a `while` loop (where `continue` jumps straight to the condition test), in a `for` loop `continue` still triggers the increment expression before the condition is re-checked.

3. C Functions & Recursion

3.1 Function Basics

- A function is a self-contained block performing a coherent task. Every C program has at least one function; if only one, it must be `main()`. Execution always begins at `main()`.
- Calling function** (suspends, passes control) vs **called function** (runs, then returns control to the exact point after the call).
- A function can be called any number of times, from any other function (even recursively from itself); functions **cannot be defined inside another function** in C (no true nested function definitions).
- Library functions** (`printf`, `scanf` — precompiled, from standard library) vs **User-defined functions**.
- Why use functions:** avoid code repetition, and enable modular/independent design, testing and readability — do not cram all logic into `main()`.

3.2 Arguments, Return Values & Prototypes

- Actual arguments** (in the calling function) vs **formal arguments/parameters** (in the called function) — their type, order, and count must match; names may differ or coincide (compiler treats them as distinct variables regardless, due to separate scope).
- Declaration styles: K&R style `calsum(x,y,z) int x,y,z; {...}` (legacy) vs ANSI style `calsum(int x, int y, int z) {...}` (modern, standard practice).
- `return` serves two purposes: (1) immediately transfers control back to the caller, (2) optionally sends a value back. A function may have multiple `return` statements; `return;` (no value) yields a garbage/unspecified value if the caller expects one.
- A function can return only **one** value directly (`return(a, b);` is invalid — the comma operator would just evaluate both and return the last, not "return two values"). To return multiple values, use **pointers** (call by address) as out-parameters instead.
- Default return type:** a C function with no explicit return type defaults to `int` (pre-C99 behavior, still commonly tested) — **TRAP:** this is why a function meant to return `float` / `double` MUST have both its **definition** and a matching **prototype declaration** state the correct return type, or the value gets silently truncated/misinterpreted as `int`.
- `void` keyword: used when a function does not return any value (and, in a parameter list, `void` means "takes no arguments").
- Function prototype** (declares name, return type, and parameter types before use — lets the compiler catch call/definition mismatches) vs **function definition** (the actual body).

3.3 Parameter Passing — Call by Value (Default in C)

- C's **only direct mechanism is Call by Value:** the *value* of each actual argument is copied into the formal argument. Changes to the formal argument inside the called function **never** affect the caller's original variable.

```
void fun(int b) { b = 60; }
void main() { int a = 30; fun(a); printf("%d", a); } // prints 30 (unchanged) - fun's b=60 is a separate copy
```

- TRAP:** C has **no true "call by reference"** like C++. You can only *simulate* reference-like behavior by explicitly passing a **pointer** (the address) — see §3.4/Ch.4. Passing `&a` is still, technically, "call by value" of an address.
- Scope rule:** every variable's default scope is local to the function where it's declared — a variable in `main()` is invisible to another function unless explicitly passed as an argument (and vice versa for returning a value out).

3.4 Recursion

- A function that calls itself is **recursive**. Every correct recursive function needs a **base case** (stops the recursion) and a **recursive case** (reduces the problem and calls itself).
- Each recursive call gets its **own stack frame** (own copies of parameters and local variables) — pushed on the call stack; frames are popped as calls return. Excessive/infinite recursion (missing/wrong base case) causes **stack overflow**.
- Direct recursion** (function A calls A) vs **Indirect/mutual recursion** (A calls B, B calls A).
- Tail recursion:** the recursive call is the LAST operation performed (nothing happens after it returns) — some compilers can optimize this into an iterative loop internally, though C gives no guarantee of this. **Non-tail (head) recursion:** work happens *after* the recursive call returns (e.g., `f = x * rec(x-1);` — the multiplication happens after `rec(x-1)` returns), so a full stack of pending operations must be kept — this is NOT tail-recursive.

Worked Trace 1 — Factorial (non-tail recursion)

```
int rec(int x) {
    if (x == 1) return 1;
    else return x * rec(x - 1);
}
```

`rec(3) → 3 * rec(2) → 3 * (2 * rec(1)) → 3 * (2 * 1) = 6`. Each call must WAIT for the inner call to return before it can multiply — classic non-tail recursion; stack depth = 3 before unwinding begins.

Worked Trace 2 — Verified recursion (doubling recurrence)

```
int x(int n) {
    if (n < 3) return 1;
    else return x(n-1) + x(n-1) + 1; // = 2*x(n-1) + 1
}
```

Trace for $n=6$: $x(2)=1 \rightarrow x(3)=2(1)+1=3 \rightarrow x(4)=2(3)+1=7 \rightarrow x(5)=2(7)+1=15 \rightarrow x(6)=2(15)+1=31$.

TRAP: this function calls `x(n-1)` TWICE (not stored/reused) — so it recomputes the same sub-values repeatedly; total calls grow **exponentially** ($O(2^n)$) despite the answer having a simple closed form, a classic naive-recursion inefficiency (same root cause as naive Fibonacci recursion).

Questions

Q1. What does a C function return by default if no return type is specified in older/legacy code?

A. void B. float C. int D. Compilation error, always

Answer: C. Why: Pre-C99 (and still commonly tested), an unspecified return type defaults to `int` — this is why returning `float / double` requires explicit declaration + matching prototype.

Q2. [Trap] Which of the following correctly returns two values from a function to its caller?

A. `return (a, b);` B. `return a; return b;` (two statements) C. Pass pointers to `a` and `b` as parameters and modify `*a`, `*b` inside the function D. It's impossible in C

Answer: C. Why: A C function can only directly return ONE value; multiple "return values" require call-by-address (out-parameters via pointers). Option A just evaluates the comma expression and returns `b`'s value alone; option B never executes past the first `return`.

Q3. [Numerical] Trace this code — what is printed?

```
int rec(int x) {
    if (x == 1) return 1;
    else return x * rec(x - 1);
}
int main() { printf("%d", rec(5)); }
```

A. 5 B. 24 C. 120 D. 15

Answer: C. Why: This is $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Q4. [Numerical] Trace this recursion for $n=6$:

```
int x(int n) {
    if (n < 3) return 1;
    else return x(n-1) + x(n-1) + 1;
}
```

A. 15 B. 21 C. 31 D. 63

Answer: C. Why: $x(2)=1$, $x(3)=3$, $x(4)=7$, $x(5)=15$, $x(6)=2(15)+1=31$ (each step doubles the previous result and adds 1).

Q5. [Trap] What is printed by the following program?

```
void junk(int i, int j) { i = i * i; j = j * j; }
int main() {
    int i = 5, j = 2;
    junk(i, j);
    printf("%d %d", i, j);
}
```

A. 25 4 B. 5 2 C. 25 2 D. 5 4

Answer: B. Why: Arguments are passed by VALUE — `junk`'s `i` and `j` are separate local copies; squaring them has no effect on `main`'s `i` and `j`, which remain 5 and 2.

Q6. Which statement about C's parameter-passing mechanism is correct?

A. C supports true call-by-reference like C++, using the `&` symbol in the function signature B. C only has call-by-value; reference-like behavior is simulated by explicitly passing pointers (addresses) C. C automatically passes arrays and structs by reference but scalars by value, with no other option D. Call-by-value and call-by-reference produce identical results in C

Answer: B. Why: Every argument in C is copied (call by value); passing a pointer just means the "value" being copied happens to be an address, letting the callee dereference it to reach the caller's original data.

Q7. A function is called "tail recursive" when:

A. It is the last function defined in the file B. The recursive call is the final action performed, with no pending computation after it returns C. It always uses a `for` loop instead of `if` D. It never has a base case

Answer: B. Why: In tail recursion, nothing happens after the recursive call returns — allowing (but not guaranteeing, in C) optimization into a loop; `return x * rec(x-1);` is NOT tail-recursive since the multiplication happens after the call returns.

Q8. [Trap] What happens when `main()` is called recursively from another function, as in:

```
void display() { printf("Hi"); main(); }
int main() { display(); }
```

A. Compilation error — `main()` cannot be called from another function B. It prints "Hi" once and terminates normally C. It repeats indefinitely, growing the call stack until a stack overflow occurs D. `main()` silently ignores the recursive call

Answer: C. Why: `main()` is a regular function in C and CAN be called recursively (a legal, if unusual, action); with no base case here, this recurses forever, consuming stack frames until the program crashes with a stack overflow.

Q9. Indirect (mutual) recursion refers to:

A. A function calling itself directly in its own body B. Function A calling function B, which in turn calls function A C. A loop disguised as a function call D. Recursion that uses no stack memory

Answer: B. Why: Indirect recursion involves two (or more) functions calling each other in a cycle, rather than a single function calling itself directly.

Q10. Why does naive recursive computation of $x(n) = x(n-1) + x(n-1) + 1$ (Q4) become inefficient for large n ?

A. It uses too much memory for a single stack frame B. It calls $x(n-1)$ twice without reusing the result, causing an exponential ($\sim O(2^n)$) number of total calls C. C cannot handle recursive functions with addition D. It is actually already optimally efficient at $O(n)$

Answer: B. Why: Since the same $x(n-1)$ is computed twice independently (not cached/memoized), the number of function calls roughly doubles at each level, giving exponential growth — the same inefficiency pattern as naive Fibonacci recursion.

4. C Arrays & Pointers

4.1 Array Basics

- Arrays store elements in **contiguous memory** (jumping from one element reaches the next directly) and are **homogeneous** (all elements same type). 0-indexed: valid indices are `0` to `size-1`.
- Size must be fixed at declaration UNLESS you initialize at the same time: `int arr[] = {10,20,30};` infers size=3 from the initializer count.
- Partial initialization** → remaining elements auto-set to 0: `int arr[5] = {1,2};` gives `{1,2,0,0,0}`.
- Uninitialized local array elements contain **garbage values** (not automatically zeroed unless explicitly initialized, or the array has static/global storage duration).
- Arrays are the underlying storage mechanism used to implement Stacks (LIFO) and Queues (FIFO) — see later DS chapters.

4.2 Pointer Basics

- A pointer is a variable that stores a memory **address**. `&` = "address-of" operator; `*` = "value-at-address" / **dereference** / indirection operator.
- Declaration `int *j;` means: "the value at the address stored in `j` is an int" — i.e., `j` is a pointer-to-int.
- `int *j = &i;` then `*j` and `i` refer to the exact same memory — reading/writing one is equivalent to the other.
- Pointer to pointer:** `int **k = &j;` — `*k` gives `j`'s value (an address), `**k` dereferences twice to reach its actual value. Chains can extend arbitrarily (pointer-to-pointer-to-pointer...).
- Pointers themselves always hold whole-number addresses regardless of the type they point to; `float *s` does NOT mean `s` holds a float — it means `s` holds the *address* of a float.

4.3 Arrays, Pointers & Address Arithmetic

- An array name **decays** to a pointer to its first element in most expressions: `arr` $\equiv$ `&arr[0]`.

```
arr[i] is EXACTLY equivalent to *(arr + i)      <- heavily tested equivalence
Loc(arr[i]) = Base_Address + i * sizeof(datatype)
```

- TRAP: Pointer arithmetic is scaled by type size**, not raw bytes — `p + 1` on an `int*` advances by `sizeof(int)` bytes (commonly 4 on a 32/64-bit system), not by 1 byte. E.g., if `int *p` holds address 1000, then `p+2` = address 1000 + 2×4 = **1008**, pointing to `arr[2]`.
- TRAP: Array name is a CONSTANT pointer** — `arr = arr + 1;` is a compile error. A pointer *variable* (`int *p = arr;`) CAN be reassigned: `p = p + 1;` is valid and moves `p` to the next element.
- Pointer subtraction works the same way in reverse: `p = p - 1;` moves backward by one element (scaled by type size).

4.4 Character Pointers & Strings

- A string is conceptually an array of characters terminated by `'\0'` (null character).
- `char str[] = "Namaste";` creates a **mutable copy** of the string in normal (writable) memory.
- `char *p = "Hello";` makes `p` point to a string **literal**, which many systems place in **read-only** memory.
- TRAP:** modifying a string literal through a `char*` (e.g., `p[0] = 'h';` on the literal-pointer version) is **undefined behavior** (often crashes) — but the same modification on the `char str[]` array copy is perfectly legal. This distinction (literal-via-pointer = read-only, array copy = mutable) is a classic C gotcha.

4.5 Call by Address (Pointers as Function Parameters)

- Passing `&variable` lets the called function dereference the pointer and modify the CALLER's actual variable — the closest C gets to "call by reference."
- A function can indirectly return **multiple** values this way (see §3.3) by writing through several pointer parameters.

Worked Trace — Call by Value vs Call by Address (mixed in one call)

```
void junk(int *i, int j) {
    *i = *i * *i;    // modifies caller's i via pointer (call by address)
    j = j * j;       // modifies only the LOCAL copy of j (call by value)
}
int main() {
    int i = 4, j = 2;
    junk(&i, j);
    printf("%d %d", i, j);
}
```

Trace: `*i` dereferences to 4, so `*i = 4*4 = 16` writes through to `main`'s `i`. `j` inside `junk` is a fresh copy (starts at 2, becomes 4), but this NEVER touches `main`'s `j`.

Output: 16 2 — **TRAP:** mixing call-by-value and call-by-address parameters in the SAME function call is a classic exam trick; you must track each parameter independently.

Questions

Q1. [Numerical] `int arr[5] = {10,20,30,40,50};` is stored starting at address 2000 (assume `int` = 4 bytes). What is the address of `arr[3]`?

A. 2003 B. 2012 C. 2008 D. 2016

Answer: B. Why: `Loc(arr[3]) = Base + 3×4 = 2000 + 12 = 2012.`

Q2. Which expression is exactly equivalent to `arr[i]`?

A. `&arr + i` B. `*(arr + i)` C. `*arr + i` D. `arr * i`

Answer: B. Why: Array subscripting `arr[i]` is defined in C to mean `*(arr + i)` — pointer arithmetic scaled by the element's type size, then dereferenced.

Q3. [Trap] Which of the following is a compile-time error?

A. `int *p = arr; p = p + 1;` B. `int arr[5]; arr = arr + 1;` C. `int *p; p = &arr[2];` D. `int *p = arr; p++;`

Answer: B. Why: `arr` (the array name) is a constant pointer and cannot be reassigned; a pointer *variable* like `p` can always be reassigned/incremented.

Q4. [Numerical] Given `int *p` holds address 5000 and `sizeof(int) == 4`, what address does `p - 2` refer to?

A. 4998 B. 4992 C. 5008 D. 4996

Answer: B. Why: Pointer arithmetic scales by type size: $5000 - 2 \times 4 = 5000 - 8 = 4992$.

Q5. [Trap] What is the likely behavior of the following code?

```
char *p = "Hello";
p[0] = 'h';
```

A. Always works fine, `p[0]` becomes 'h' B. Undefined behavior / likely crash — string literals are commonly stored in read-only memory C. Compile-time error, guaranteed D. `p` automatically becomes a new mutable copy

Answer: B. Why: `p` points to a string literal, which many implementations place in read-only memory; writing through it is undefined behavior. Using `char p[] = "Hello";` instead would create a safely mutable copy.

Q6. [Numerical/Trap] Trace this program's output:

```
void junk(int *i, int j) {
    *i = *i * *i;
    j = j * j;
}
int main() {
    int i = 4, j = 2;
    junk(&i, j);
    printf("%d %d", i, j);
}
```

A. 16 4 B. 4 2 C. 16 2 D. 4 4

Answer: C. Why: `i` is passed by address (`*i` modifies the caller's variable: $4 \times 4 = 16$), but `j` is passed by value (only the local copy is squared; caller's `j` stays 2).

Q7. Declaring `int **k;` means:

A. `k` is an array of 2 integers B. `k` is a pointer to an integer pointer (pointer-to-pointer) C. `k` is a double-precision integer D. Invalid syntax in C

Answer: B. Why: Each `*` adds one level of indirection; `int **k` holds the address of an `int*` variable, requiring two dereferences (`**k`) to reach the underlying `int` value.

Q8. Why can a C function only directly `return` one value, yet a function like `areaperi(int r, float *a, float *p)` can effectively produce two results?

A. It cannot — this is impossible in C B. It uses call-by-address: the function writes results into the caller's variables via pointers, rather than using `return` C. `return` in C accepts a comma-separated list D. C automatically packs multiple values into a struct behind the scenes

Answer: B. Why: By passing pointers to `area` and `perimeter`, the function dereferences and assigns to `*a / *p`, directly modifying the caller's variables — sidestepping the one-value-per-return limitation.

Q9. In `int arr[4] = {7};`, what are the values of `arr[1]`, `arr[2]`, `arr[3]` ?

A. Garbage values B. 7, 7, 7 C. 0, 0, 0 D. Undefined/compile error

Answer: C. Why: Partial initialization sets all remaining, un-listed elements to 0 automatically — only `arr[0]` is explicitly set to 7.

Q10. A pointer variable `p` and an array name `arr` both can appear as `arr` or `p` in expressions referring to "the address of the first element." What is the key functional difference between them?

A. There is no difference whatsoever B. `arr` is a constant pointer (cannot be reassigned); `p` is a regular variable (can be reassigned to point elsewhere) C. `p` can only point to arrays, `arr` can point to any variable D. `arr` uses more memory than `p`

Answer: B. Why: The array name is fixed at compile time to the array's base address; a pointer variable is free to be redirected to any compatible address at runtime.

5. C Storage Classes, Structures, Unions & Enums

5.1 Storage Classes

Every C variable has both a **type** and a **storage class**. The storage class determines: where it's stored, its default initial value, its scope, and its lifetime.

Storage Class	Stored In	Default Value	Scope	Lifetime
auto (default)	Memory (stack)	Garbage	Local to block	Only while control is inside the block
register	CPU register (if available)	Garbage	Local to block	Only while control is inside the block
static	Memory	Zero	Local to block (or file, for global statics)	Entire program — value persists across calls
extern	Memory	Zero	Global (across files)	Entire program

- **TRAP:** You **cannot take the address (&) of a register variable**, since it may not have a memory address at all (it may live only in a CPU register). Also, not every type fits in a register (e.g., `float` / `double` on a 16-bit-register machine) — the compiler silently falls back to `auto` behavior in that case.
- **TRAP:** `static`'s defining exam behavior is that it is initialized **only once**, and **retains its value between function calls** — a `static` local variable used as a counter keeps incrementing across repeated calls, unlike an `auto` local which resets to its initial value every call.

Worked Trace — auto vs static

```
void increment_auto() { int i = 1;      printf("%d ", i); i++; }
void increment_static() { static int i = 1; printf("%d ", i); i++; }
// Called 3 times each:
increment_auto(); increment_auto(); increment_auto(); // prints: 1 1 1
increment_static(); increment_static(); increment_static(); // prints: 1 2 3
```

Scope Rule — Nested Blocks

```
main(){
  auto int i = 1;
  { auto int i = 2;
    { auto int i = 3; printf("%d ", i); } // prints 3 (innermost i)
    printf("%d ", i);                    // prints 2
  }
  printf("%d", i);                      // prints 1
}
// Output: 3 2 1 (each block's i is a completely separate variable)
```

- Local variable **always takes preference** over a global variable of the same name within the block where it's declared (name shadowing) — **TRAP:** this is a very common "what does this print" question.
- `extern` is a **declaration** (no memory reserved) vs a variable **definition** (memory reserved) — a variable can be *declared* many times but *defined* only once.
- A `static` global (file-scope) variable behaves like `extern` (zero default, whole-program lifetime) EXCEPT its scope is limited to the file it's declared in — it is NOT visible to other files, unlike a true `extern` / global.

5.2 Static vs Dynamic Scoping

- **Static (Lexical) Scoping** — a variable reference is resolved by the program's **text structure** (nesting of blocks/functions as written), independent of the runtime call sequence. Resolved at **compile time**. **C, C++, Java, Python all use static scoping.**
- **Dynamic Scoping** — a variable reference is resolved by the **most recent active call on the call stack** at runtime, not by lexical nesting. Resolved at **runtime**. Rare in modern languages (some Lisp dialects, `local` in Perl).

Worked Trace — proves C is statically scoped

```
int x = 10;
int f(){ return x; }           // f is defined at FILE scope -> can only see global x
int g(){ int x = 20; return f(); } // g's local x=20 is NOT visible inside f()
int main(){ printf("%d", g()); }
// Output: 10 (NOT 20 - f() resolves x lexically to the global, ignoring g's local x
//           even though g() is the one that called f())
```

TRAP: If this same code pattern ran under **dynamic scoping**, `f()` would instead pick up `g`'s most-recently-active `x = 20` from the call stack, printing **20**. The fact that real C prints **10** is the proof that C is lexically/statically scoped — a favorite conceptual GATE/ISRO question.

Aspect	Static Scoping	Dynamic Scoping
Resolved by	Program text (lexical nesting)	Call stack at runtime
Binding time	Compile time	Runtime
Speed	Faster (known at compile time)	Slower (stack traversal)
Predictability	High — read the source to know	Low — depends on execution path
Supports closures?	Yes	No

5.3 Structures

- A **structure** groups **heterogeneous** (possibly different-type) members under one name — contrast with an array, which groups **homogeneous** (same-type) elements.
- Declaration only defines the **form/type** — it reserves **no memory** until a variable of that struct type is declared.

- Access: **dot** (`.`) operator for a struct variable (`b1.pages`), **arrow** (`->`) operator for a struct pointer (`p->pages` , equivalent to `(*p).pages`).
- Members are stored in **contiguous** memory, in declaration order — BUT the compiler may insert **padding bytes** to align each member on its natural boundary.

Worked Trace — Structure Padding

```
struct Example { char c; int i; }; // naive size = 1 + 4 = 5 bytes
// Actual sizeof(struct Example) is typically 8 bytes on a system with 4-byte int alignment:
// c occupies offset 0 (1 byte), then 3 PADDING bytes (offsets 1-3) so that
// i can start at a 4-byte-aligned offset (offset 4), i occupies offsets 4-7.
```

TRAP: `sizeof(struct)` is often **larger** than the sum of its members' sizes, because of alignment padding — never assume struct size = sum of member sizes.

- Whole-structure **assignment** is allowed directly (`e3 = e2;` copies every member) — no need to copy member-by-member.

- **Array of structures** (`struct book b[100];`) is the standard way to store many records of the same struct type.

5.4 Union vs Structure

- A **union** looks like a struct syntactically, but **all members share the SAME memory location** — only one member holds a valid value at any time; writing to one member overwrites any other member's data.
- `sizeof(union)` = size of its **largest** member (not the sum) — direct contrast with a struct, whose size is roughly the **sum** of its members (plus padding).

Feature	Structure	Union
Memory	Each member gets its own space (sum, plus padding)	All members share one space
Size	~ sum of member sizes (+ padding)	Size of the largest member
Simultaneously valid members	All	Only one at a time
Use case	Represent a record with multiple independent fields	Represent one value that can be interpreted as different types

TRAP: In a union, setting one member and then reading a *different* member gives **garbage/reinterpreted bits**, not the value you set — this is by design, not a bug.

5.5 Enums

- `enum` defines a set of **named integer constants**. By default, the first name = **0**, and each subsequent name = **previous value + 1**.
- Custom values can be assigned to any constant; unassigned ones after a custom value continue incrementing from THAT value (not from 0).

```
enum Number { ONE = 5, TWO, THREE, FOUR = 20, FIVE };
// ONE=5, TWO=6, THREE=7, FOUR=20, FIVE=21
```

- Enum constants are internally just **integers** — `printf("%d", enumVar)` works directly.
- Limitations: fixed at compile time (can't add new constants at runtime), only integer-valued, not suited for large/dynamic value sets.

Questions

Q1. [Trap] What does the following program print?

```
void f() { static int i = 1; printf("%d ", i); i++; }
int main() { f(); f(); f(); return 0; }
```

A. 1 1 1 B. 1 2 3 C. 0 1 2 D. 3 3 3

Answer: B. Why: `static` initializes `i` only once; its value persists across calls, incrementing each time: 1, then 2, then 3.

Q2. Which storage class has a default (unassigned) value of garbage AND cannot have its address taken with `&` ?

A. auto B. static C. extern D. register

Answer: D. Why: `register` variables may live in a CPU register (which has no memory address), so `&` on them is illegal; both auto and register default to garbage, but only register has the no-address restriction.

Q3. [Trap] What is the output?

```
int x = 10;
int f(){ return x; }
int g(){ int x = 20; return f(); }
int main(){ printf("%d", g()); return 0; }
```

A. 10 B. 20 C. Compile error D. Undefined behavior

Answer: A. Why: C uses static (lexical) scoping — `f()` is defined at file scope and can only see the global `x=10` ; it has no visibility into `g`'s local `x=20` regardless of who called it.

Q4. [Numerical] Given `struct S { char c; int i; };` on a system where `int` requires 4-byte alignment, what is the most likely value of `sizeof(struct S)` ?

A. 5 B. 8 C. 4 D. 1

Answer: B. Why: `char c` takes 1 byte, then 3 padding bytes are inserted so `int i` starts at a 4-byte-aligned offset — total 8 bytes, not the naive 1+4=5.

Q5. [Trap] Given `union U { int i; float f; char str[20]; };`, what is `sizeof(union U)` ?

A. Sum of all members B. Size of the largest member (20, from `str`) C. Size of `int` only D. 0

Answer: B. Why: A union allocates only enough space for its largest member since all members share the same memory.

Q6. What are the values of RED, GREEN, BLUE in `enum Color { RED, GREEN = 20, BLUE };` ?

A. 0, 20, 21 B. 0, 20, 1 C. 1, 20, 21 D. 0, 1, 2

Answer: A. Why: RED defaults to 0 (no prior value); GREEN is explicitly set to 20; BLUE continues incrementing from the previous value, becoming 21.

Q7. Accessing a member through a structure POINTER `p` uses which operator?

A. `.` B. `->` C. `::` D. `&`

Answer: B. Why: The arrow operator `p->member` is shorthand for `(*p).member`, used specifically when you have a pointer to a struct rather than the struct itself.

Q8. [Trap] Which of these correctly describes dynamic scoping (NOT used by standard C)?

A. Variable binding resolved by program text at compile time B. Variable binding resolved by the most recent active call on the runtime call stack C. Identical to static scoping for all practical purposes D. Used by C, C++, Java, and Python

Answer: B. **Why:** Dynamic scoping resolves a name using the call stack at runtime (so the same code can behave differently depending on who called whom), whereas C's static scoping resolves purely from lexical/text structure at compile time.

Q9. A local variable declared `auto` inside a block loses its value when:

A. The program ends B. Control exits the block in which it was declared C. Another function is called D. It never loses its value

Answer: B. **Why:** An `auto` variable's lifetime is scoped to its enclosing block — once control leaves that block, the variable (and its value) is gone.

6. Dynamic Memory Allocation, Macros & File Handling

6.1 Dynamic Memory Allocation (stdlib.h)

Function	Args	Initializes memory?	Notes
<code>malloc(size)</code>	total bytes	No (garbage)	Single argument; returns <code>void*</code> , or <code>NULL</code> on failure
<code>calloc(n, size)</code>	count, size-per-element	Yes, to 0	Two arguments; slower than <code>malloc</code> due to zeroing
<code>realloc(ptr, newsize)</code>	existing pointer, new size	Preserves old data; new region uninitialized	May move the block — always reassign the returned pointer
<code>free(ptr)</code>	pointer to release	—	Deallocates; does NOT set <code>ptr</code> to <code>NULL</code> automatically

```
int *p1 = (int*) malloc(10 * sizeof(int)); // 10 ints, garbage values
int *p2 = (int*) calloc(10, sizeof(int)); // 10 ints, all zero
```

- **TRAP: Memory leak** = allocated memory that is never `free()`'d — the pointer may go out of scope, making that memory permanently unreachable (and un-freeable) until the program exits.
- **TRAP: Dangling pointer** = using a pointer *after* it has been `free()`'d (or after the memory it pointed to has otherwise expired) — this is undefined behavior (may appear to "work" sometimes, may crash, may silently corrupt data). Always set a pointer to `NULL` after freeing it as a defensive habit.
- Static allocation (ordinary variables, fixed array sizes) is decided at **compile time** on the **stack**; dynamic allocation (`malloc/calloc/realloc`) is decided at **runtime** on the **heap** — heap memory persists until explicitly freed, unlike stack memory which is automatically reclaimed when a block/function exits.

6.2 Static vs Dynamic Scoping

(See Ch.5 §5.2 for the full treatment — C uses **static/lexical** scoping exclusively; a variable reference is resolved by the program's *text* structure, not by the runtime call chain.)

6.3 Macros (`#define`)

- A macro is a **preprocessor** directive: pure **textual substitution** performed *before* compilation — NOT a function call, so there is **no type checking** and **no call overhead**.
- **Object-like macro**: `#define MAX 100` — simple constant substitution.
- **Function-like macro**: `#define SQUARE(x) ((x)*(x))` — takes parameters, but substitutes literally (text-for-text), rather than evaluating like a real function.
- Benefits: faster (no call overhead), but can cause **code bloat** if used heavily (each call site gets its own copy of the expanded code, unlike a function which has one shared body).

TRAP — Unparenthesized Macro Expansion

```
#define abc(x) x*x
int a = 64 / abc(4);
// Expands LITERALLY to: a = 64 / 4*4;
// / and * have EQUAL precedence, left-to-right: (64/4)*4 = 16*4 = 64
// a = 64 -- NOT 4, which is what you'd get if abc(4) were evaluated as a real function first!
```

Always wrap BOTH the macro parameter AND the whole expression in parentheses: `#define SQUARE(x) ((x)*(x))` to avoid precedence-driven surprises like this.

TRAP — Macros Don't Substitute Inside String Literals

```
#define mno pqr
printf("mno"); // Prints: mno (NOT pqr — the preprocessor does not touch text inside quotes)
```

Token-Pasting Operator `##`

```
#define fun(i, j) i##j
int f1 = 10, f12 = 20;
printf("%d", fun(f1, 2)); // fun(f1,2) -> f1##2 -> pastes into identifier "f12"
// f12 = 20, so this prints: 20
```

TRAP — Macro-Redefining a Keyword

```
#define int char
int a = 10, *p = &a;
*p = 20;
printf("%d %d", sizeof(a), sizeof(p));
// After preprocessing, EVERY token "int" becomes "char":
// char a = 10, *p = &a; (p is now a "char *")
// sizeof(a) = 1 (sizeof(char))
// sizeof(p) = size of a pointer (architecture-dependent: commonly 4 on a 32-bit system, 8 on 64-bit)
// Output (32-bit assumption): 1 4
```

This shows macros perform **blind textual replacement** of the exact token — even a language keyword can be "redefined" this way (a favorite ISRO/GATE trick question).

6.4 File Handling in C

- Purpose: persist data beyond program execution (unlike variables, which vanish when the program ends).
- **Text files** (human-readable, line-based) vs **Binary files** (raw bytes, not human-readable, more compact/faster for structured data).

Core Workflow

```
FILE *fp = fopen("data.txt", "r"); // 1. Open – ALWAYS check fp != NULL before use
if (fp == NULL) { /* handle error */ }
// 2. Perform operations (read/write/append)
fclose(fp); // 3. Close – flushes buffers, releases resources
```

fopen() Modes

Mode	Meaning	If file doesn't exist
r	Read only	Returns NULL (fails)
w	Write (truncates /overwrites existing content)	Creates new file
a	Append (writes go to the end)	Creates new file
r+	Read + Write, starts at beginning	Fails
w+	Read + Write, truncates existing content	Creates new file
a+	Read + Append	Creates new file

- Append `b` to any mode (e.g. `rb`, `wb+`) for **binary mode**.
- TRAP:** `"w"` and `"w+"` **destroy existing file content immediately upon opening** — a very common accidental-data-loss trap. Use `"a"` / `"a+"` if you want to preserve existing content.

Read/Write Function Pairs

Write	Read	Purpose
<code>fprintf()</code>	<code>fscanf()</code>	Formatted (like <code>printf</code> / <code>scanf</code> but to/from a file)
<code>fputs()</code>	<code>fgets()</code>	Whole line
<code>fputc()</code>	<code>fgetc()</code>	Single character
<code>fwrite()</code>	<code>fread()</code>	Raw bytes (binary files)

- `feof(fp)` — tests for End-Of-File; important to check while reading in a loop to avoid reading past the end (undefined behavior).
- `fseek(fp, offset, pos)` — moves the file pointer; `pos` is a reference point (`SEEK_SET` =start, `SEEK_CUR` =current, `SEEK_END` =end), `offset` can be negative.
- `ftell(fp)` — returns the current file-pointer position (byte offset from start).
- `rewind(fp)` — resets the file pointer to the very beginning (equivalent to `fseek(fp, 0, SEEK_SET)`).

Questions

Q1. [Trap] What is the output?

```
#define abc(x) x*x
int main(){ int a = 64/abc(4); printf("%d", a); return 0; }
```

A. 4 B. 16 C. 64 D. 1

Answer: C. Why: Textual expansion gives `64/4*4`; `/` and `*` share precedence and associate left-to-right: $(64/4)4 = 164 = 64$.

Q2. [Trap] What is the output?

```
#define mno pqr
int main(){ printf("mno"); return 0; }
```

A. pqr B. mno C. Compile error D. Empty output

Answer: B. Why: The preprocessor never substitutes tokens that appear inside a string literal — `"mno"` is printed exactly as written.

Q3. Which dynamic memory function BOTH allocates memory AND initializes it to zero?

A. `malloc()` B. `calloc()` C. `realloc()` D. `free()`

Answer: B. Why: `calloc(n, size)` is the only one of the three allocation functions that zero-initializes the memory; `malloc` leaves it uninitialized (garbage).

Q4. [Trap] Using a pointer after calling `free()` on it, without setting it to NULL, is called a:

A. Memory leak B. Dangling pointer C. Segmentation table D. Buffer overflow

Answer: B. Why: A dangling pointer refers to freed (deallocated) memory; using it afterward is undefined behavior — distinct from a memory leak, which is forgetting to free memory at all.

Q5. Which `fopen()` mode will DESTROY the existing contents of a file the moment it is opened?

A. `"r"` B. `"a"` C. `"w"` D. `"a+"`

Answer: C. Why: Mode `"w"` truncates (erases) any existing file content immediately on open; `"a"/"a+"` preserve existing content and only add to the end; `"r"` doesn't modify content at all (and fails if the file doesn't exist).

Q6. [Numerical] Given `int f1 = 10, f12 = 20;` and `#define fun(i,j) i##j`, what does `fun(f1, 2)` evaluate to?

A. 10 B. 20 C. 12 D. Compile error

Answer: B. Why: The `##` operator pastes tokens together: `fun(f1,2)` becomes the single identifier `f12`, whose value is 20.

Q7. Which function moves the file pointer back to the very beginning of a file?

A. `fseek()` only B. `ftell()` C. `rewind()` D. `fEOF()`

Answer: C. Why: `rewind(fp)` resets the position to the start; it's equivalent to `fseek(fp, 0, SEEK_SET)` but simpler to use for this specific purpose.

Q8. [Trap] What is a key difference between a macro and a real C function?

A. Macros support type checking, functions don't B. Macros are expanded textually at compile time with no call overhead; functions involve an actual call with type checking C. Functions are always faster than macros D. There is no meaningful difference

Answer: B. Why: Macros are pure preprocessor text substitution — no type checking, no call overhead, but can cause code bloat; functions are compiled once and called, with full type checking, at some (usually small) call overhead.

Q9. [Trap] After `#define int char`, what does `int a = 10, *p = &a;` actually declare (post-preprocessing)?

A. `a` as an int, `p` as an int pointer B. `a` as a char, `p` as a char pointer C. A compile error D. No change — macros can't redefine keywords

Answer: B. Why: The preprocessor performs blind textual replacement of every occurrence of the token "int" with "char" before compilation even begins — this includes keyword tokens, producing `char a = 10, *p = &a; .`

7. OOP with C++ — Classes, Objects, Constructors & Overloading

7.1 C vs C++ and the OOP Paradigm

- **C** (Dennis Ritchie, 1972) is procedural/function-oriented. **C++** (Bjarne Stroustrup, 1985) extends C by adding Object-Oriented Programming while keeping all of C's features — so C++ supports BOTH procedural and OOP styles.
- **Middle-level language:** C++ gives low-level features (direct memory access via pointers, fast execution) AND high-level features (functions, portability, readability) in one language.

Feature	Structured (Procedural) Programming	Object-Oriented Programming
Organized around	Functions/procedures	Objects (data + methods bundled)
Approach	Top-down	Bottom-up
Data security	Weak — data is often global/shared	Strong — encapsulation restricts access
Reusability	Harder	Easier (inheritance, classes)
Example languages	C	C++, Java, Python, C#

- Key OOP pillars introduced by C++ (none of these exist in plain C): **Classes & Objects, Inheritance, Polymorphism, Encapsulation, Abstraction, Function/Operator Overloading, Templates.**

7.2 Classes & Objects

- **Class** = user-defined data type / blueprint bundling data members + member functions. **Object** = an instance of a class; memory is allocated only when an object is created, and each object has its own independent copy of the data members (but shares the same member function code).
- Analogy: Class = house blueprint, Object = an actual house built from it.

```
class Student {
    int roll;
public:
    void display() { cout << roll; }
};
Student s1, s2; // two independent objects
```

- **this pointer:** implicitly passed to every non-static member function; holds the address of the CURRENT object. Used to (a) resolve name conflicts between a data member and a same-named parameter, (b) return the current object via `return *this;` (enables chained calls like `obj.set(1).set(2)`).
- **Scope Resolution Operator (::)** lets C++ access a global variable that is hidden ("shadowed") by a local variable of the same name (`::x` reaches the global `x`) — this is a problem C had NO fix for; C++ solved it with `::`.

7.3 struct vs class in C++

Feature	struct	class
Default member access	public	private
Default inheritance mode	public	private
Typical use	Simple data grouping	Full OOP (encapsulation-focused)

- **TRAP:** In C++ (unlike plain C), a `struct` CAN have member functions, constructors, and destructors too — the ONLY hard default difference from `class` is the default access specifier (public vs private).

7.4 Reference Variables

- A reference (`int &ref = x;`) is an **alias** for an existing variable — it does NOT get its own memory; it must be initialized at declaration, and it can **never be re-seated** to refer to a different variable afterward.
- **TRAP:** A pointer CAN be reassigned to point elsewhere later; a reference CANNOT — this is the key pointer-vs-reference distinction.

7.5 Inline Functions

- Requests the compiler to substitute the function's code directly at the call site instead of a real function call → removes call overhead, best for small, frequently-called functions.
- Functions **defined inside a class body are implicitly inline** by default.
- The compiler MAY ignore the inline request (it is only a *request*, never a guarantee) when the function contains loops, a switch, is recursive, contains goto, or is simply too large/complex.

7.6 Static Members

- **Static data member:** belongs to the CLASS, not to any object — only ONE copy exists no matter how many objects are created; all objects share it, and a change by one object is visible to all. Must be defined/initialized OUTSIDE the class using `::` (e.g. `int Student::count = 0;`).
- **Static member function:** can be called via the class name (`ClassName::func()`) without any object; can ONLY access other static members directly (no `this` pointer exists for it) — it CANNOT touch non-static/instance data or call non-static member functions directly.
- **TRAP:** A static member function has no `this` pointer — trying to access an ordinary (non-static) data member inside it is a compile error.

7.7 Friend Functions & Friend Classes

- A **friend function** is a non-member function explicitly granted access to a class's private/protected members (declared with `friend` inside the class, defined outside it). Commonly needed for operator overloading that must touch both operands' private data.

- A **friend class**: ALL member functions of the friend class get access to the granting class's private/protected members.
- Key friendship rules: **NOT mutual** (A befriends B doesn't mean B befriends A), **NOT inherited**, **NOT transitive**.

Friend Function	Friend Class
Only ONE specific function gets access	ALL member functions of that class get access
<code>friend return_type f(...);</code>	<code>friend class ClassName;</code>

7.8 Constructors & Destructors

- **Constructor**: special member function, SAME NAME as the class, auto-invoked on object creation, NO return type (not even void), can be overloaded, CANNOT be called explicitly via `.`, `::`, and CANNOT be virtual.
- If the programmer defines ANY constructor, the compiler does **not** generate its own default (no-argument) constructor — **TRAP**: this means providing only a parameterized constructor and then trying `MyClass obj;` (no args) will fail to compile.
- Types: **Default** (no args) · **Parameterized** (init via arguments) · **Copy constructor** `ClassName(const ClassName &obj)` — invoked automatically when one object is initialized from another (`Temp t2 = t1;`, pass-by-value, return-by-value).
- **Shallow copy vs Deep copy** (the single most tested constructor concept):

	Shallow Copy	Deep Copy
What's copied	The pointer VALUE (address) only	New memory allocated + actual pointed-to DATA copied
Result	Both objects point to the SAME memory	Each object has independent memory
Risk	One object's change affects the other; double-free on destruction	None — fully independent
Who provides it	Compiler's default copy constructor (automatic)	Programmer must write a custom copy constructor

- **TRAP**: The **compiler-generated default copy constructor always performs a shallow copy** — if a class holds a raw pointer to dynamically allocated memory, you MUST write your own deep-copy constructor or risk both objects freeing the same memory.
- **Destructor**: name = `~ClassName`, no arguments (so it can NEVER be overloaded), no return type, auto-invoked when an object is destroyed/goes out of scope, executes in REVERSE order of construction, and (unlike constructors) a destructor **CAN be declared virtual** (essential in inheritance — see Ch.8).

7.9 Function Overloading vs Operator Overloading

- **Function Overloading**: same name, different parameter list (count/type/order) — resolved at **compile time** = "Compile-Time / Static Polymorphism."
- **Operator Overloading**: redefining an existing operator to work with a user-defined (class) type, via a member function or a friend function.

Rules: only EXISTING operators can be overloaded; no new operator can be invented; at least one operand must be a user-defined object; operator precedence/associativity/arity (number of operands) CANNOT be changed; built-in-type-only operations cannot be overloaded.

- Operators that can **NEVER** be overloaded: `.` (member access), `::` (scope resolution), `sizeof`, `?:` (ternary), `.*` (pointer-to-member).
- Operators that **MUST** be overloaded as a member function (friend function NOT allowed): `=` (assignment), `[]` (subscript), `()` (function call), `->` (member access through pointer).
- **TRAP**: Students often assume ANY operator can be overloaded via a friend function — the four operators above are a hard exception and must be true class members.

Questions

Q1. [Trap] What is the ONLY default difference between a `struct` and a `class` in C++?

A. struct cannot have member functions B. struct members default to public, class members default to private C. struct cannot have constructors D. struct cannot be inherited from

Answer: B. **Why:** In C++, struct and class are functionally identical except for their default access specifier (public vs private) and default inheritance mode.

Q2. A reference variable differs from a pointer in that:

A. A reference can be re-seated to refer to another variable; a pointer cannot B. A reference must be initialized at declaration and can never be re-seated; a pointer can be reassigned freely C. A reference occupies its own separate memory like a pointer does D. There is no difference

Answer: B. **Why:** References are permanent aliases fixed at initialization; pointers are independent variables that can be redirected to point elsewhere at any time.

Q3. [Trap] Which of these correctly describes a static member function's capability?

A. It can access both static and non-static members directly B. It can access ONLY static members directly, since it has no `this` pointer C. It requires an object to be called D. It is automatically virtual

Answer: B. **Why:** A static member function is not tied to any specific object (no `this` pointer), so it can only directly touch other static members.

Q4. What does the compiler-generated DEFAULT copy constructor perform when a class contains a raw pointer member?

A. A deep copy automatically B. A shallow copy — copying the pointer's address, not the pointed-to data C. It refuses to compile D. It converts the pointer to a reference

Answer: B. **Why:** The default copy constructor copies member values as-is; for a pointer member, that means copying the address, leaving both objects pointing at the same memory (shallow copy) — a classic source of double-free bugs.

Q5. [Trap] Which of the following operators CANNOT be overloaded in C++?

A. `+` B. `==` C. `sizeof` D. `[]`

Answer: C. **Why:** `sizeof`, `.`, `::`, `?:`, and `.*` can never be overloaded; `+`, `==`, and `[]` all can be.

Q6. Which operator MUST be overloaded only as a member function, never as a friend function?

A. `+` B. `-` C. `[]` D. `*`

Answer: C. **Why:** `=`, `[]`, `()`, and `->` require member-function overloading — this is a hard C++ rule.

Q7. Function Overloading is resolved:

A. At compile time, based on the number/type of arguments B. At runtime, based on the object's actual type C. Only when a virtual keyword is used D. Randomly by the linker

Answer: A. **Why:** Function overloading is Compile-Time (Static) Polymorphism — the compiler picks the matching overload based on argument signature before the program ever runs.

Q8. [Trap] If a class defines only a parameterized constructor `MyClass(int x)` and no other constructor, what happens with `MyClass obj;`?

A. It compiles fine using an implicit default constructor B. Compile error — no matching default constructor exists C. `obj` is created with `x=0` D. `obj` is created with

garbage x

Answer: B. Why: Once ANY user-defined constructor exists, the compiler stops auto-generating a default (no-arg) constructor — `MyClass obj;` now has no matching constructor and fails to compile.

Q9. A friend function is best described as:

A. A member function with private access B. A non-member function explicitly granted access to a class's private/protected members C. Automatically inherited by derived classes D. A function that grants access mutually between two classes

Answer: B. Why: Friend functions are NOT members, and friendship is neither mutual, inherited, nor transitive — it's a one-way, explicit grant.

Q10. [Code trace] What does this print?

```
class Counter {
    static int count;
public:
    Counter() { count++; }
    static int getCount() { return count; }
};
int Counter::count = 0;
int main() {
    Counter a, b, c;
    cout << Counter::getCount();
}
```

A. 0 B. 1 C. 3 D. Compile error

Answer: C. Why: `count` is static — shared by all objects; each of the 3 constructor calls (a, b, c) increments the SAME copy, so it reaches 3.

8. OOP with C++ — Inheritance, Polymorphism & Templates

8.1 Inheritance Basics & Access-Specifier Interaction

- **Inheritance**: a derived (child/sub) class acquires the members of a base (parent/super) class — models an **IS-A** relationship, promotes reusability.
- Access specifiers: **public** (accessible anywhere), **private** (only inside the same class — default for a class), **protected** (inside the same class AND derived classes, but not from outside via an object).
- How a base member's access level changes under each inheritance mode:

Base Member	Private Inheritance	Protected Inheritance	Public Inheritance
private	Not Inherited (NI)	Not Inherited (NI)	Not Inherited (NI)
protected	becomes Private	stays Protected	stays Protected
public	becomes Private	becomes Protected	stays Public

- **TRAP**: A base class's `private` members are **NEVER directly inherited/accessible** in the derived class under ANY inheritance mode — only `protected` / `public` base members pass through (with visibility possibly narrowed).

8.2 Types of Inheritance

Type	Structure
Single	1 derived class ← 1 base class
Multiple	1 derived class ← 2+ DIFFERENT base classes
Hierarchical	2+ derived classes ← 1 common base class
Multilevel	Chain: A ← B ← C (B is both a derived class of A and a base class of C)
Hybrid	Combination of two or more of the above (e.g. Multiple + Multilevel)

- Note: constructors are **never inherited** — each class must define/inherit-call its own.

8.3 Diamond Problem vs Ambiguity Problem (classic trap pair)

	Ambiguity Problem	Diamond (Hybrid) Problem
Cause	Two UNRELATED base classes happen to have a same-named member	The SAME base class is reached through two different inheritance paths (multiple + hierarchical combined)
Effect	Naming conflict — compiler can't tell which member you mean	Derived class ends up with 2 DUPLICATE copies of the common ancestor's members
Fix	Scope Resolution Operator (<code>Base::member</code>) or override in derived class	Virtual Inheritance (<code>class B : virtual public A</code>) — ensures only ONE shared copy of A

- **TRAP**: Do not conflate these — Ambiguity is about **name conflicts** between unrelated bases (fixed with `::`); Diamond Problem is about **duplicate copies of a shared ancestor** (fixed with `virtual` inheritance). Using `::` does NOT fix the Diamond Problem's duplication.

```
class A {};
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {}; // D has exactly ONE A subobject (thanks to `virtual`)
```

8.4 Constructor/Destructor Order in Inheritance

```
Object creation:   Base class constructor -> Derived class constructor
Object destruction: Derived class destructor -> Base class destructor (exact REVERSE order)
```

- If the base class has ONLY a parameterized constructor (no default), the derived class's constructor **must** explicitly forward arguments via an initializer list: `Derived(int y) : Base(y) { }` — omitting this is a compile error ("no matching function for call to Base()").

8.5 Object Slicing

- Assigning a derived-class object to a base-class object **by value** copies ONLY the base-class portion — the derived-specific members are "sliced off" and lost.

```
class Employee { public: string name; };
class Manager : public Employee { public: int teamSize; };
Manager m; Employee e = m; // Object Slicing: e.teamSize doesn't exist, only e.name survives
```

- **TRAP**: Object slicing silently **disables runtime polymorphism** for that copy (since only the base part remains) — this is why polymorphic objects are almost always accessed via base-class **pointers/references**, never assigned by value.

8.6 Polymorphism — Compile-Time vs Runtime

	Compile-Time (Static) Polymorphism	Runtime (Dynamic) Polymorphism
Achieved via	Function Overloading, Operator Overloading	Function Overriding + Virtual Functions
Resolved	At compile time (by the compiler)	At execution time (dynamic dispatch)
Also called	Early Binding	Late Binding
Needs <code>virtual</code> ?	No	Yes (at least one virtual function)
Speed	Faster	Slightly slower (vtable lookup overhead)

- **Function Overloading** (same name, different params, same class, compile-time) vs **Function Overriding** (same name + same params + same return type, Base–Derived relationship, requires the base function to be `virtual`, resolved at runtime) — a frequently confused pair.
- **Function Overriding conditions:** inheritance relationship + identical signature + base function marked `virtual` + accessed via a base-class pointer/reference.

8.7 Virtual Functions

- A `virtual` base-class function lets a **derived class override it**; when called through a **base-class pointer/reference**, the DERIVED class's version actually executes — this is the entire mechanism behind runtime polymorphism (dynamic dispatch via the vtable).

```
class Animal { public: virtual void sound() { cout << "..."; } };
class Dog : public Animal { public: void sound() override { cout << "Bark"; } };
Animal *ptr = new Dog();
ptr->sound(); // prints "Bark" – the DERIVED version, even though ptr's static type is Animal*
```

- Rules: must be a member function (not static, not a constructor); once declared `virtual` in the base, it **stays virtual** in every derived override automatically (no need to repeat the keyword, though `override` is good practice); **destructors CAN be virtual** (and should be, in a polymorphic base class); **friend functions cannot be virtual** (they aren't class members).
- **TRAP: Virtual Destructor** — `Base *ptr = new Derived(); delete ptr;` — if `~Base()` is NOT virtual, only the base destructor runs, leaking any resources the derived part allocated. ALWAYS make a base class destructor virtual if the class will be used polymorphically through base pointers.

8.8 Pure Virtual Functions & Abstract Classes

- **Pure virtual function:** `virtual void show() = 0;` — no implementation in the base class.
- A class with **at least one** pure virtual function becomes an **Abstract Class** — it CANNOT be instantiated directly, and exists only to define a common interface/contract.
- A derived class becomes **Concrete** (instantiable) only once it implements **ALL** inherited pure virtual functions — if even one remains unimplemented, the derived class is ALSO still abstract.
- Constructors can never be virtual (pure or otherwise); a base-class pointer/reference CAN still point to a derived (concrete) object even though the base itself is abstract.

Virtual Function	Pure Virtual Function
Has a body in the base class	No body in base (= 0)
Overriding is optional	Overriding is MANDATORY (else derived stays abstract)
Base class is concrete	Base class becomes Abstract

8.9 Templates & STL

- **Templates** write one generic function/class that works across data types without duplicating code: **Function templates** (`template<class T> T add(T a, T b) { ... }`) and **Class templates**.
- **STL (Standard Template Library):** ready-made generic Containers (vector, list, stack, queue, map, set), Algorithms (sort, search), and Iterators (traverse containers) — the 3 core STL components.
- **vector** = STL's dynamic array (`#include <vector>`) — automatically grows/shrinks, unlike a plain C-style array whose size is fixed at declaration.

Questions

Q1. [Code trace] What is printed?

```
class Base {
public:
    Base() { cout << "B"; }
    ~Base() { cout << "b"; }
};
class Derived : public Base {
public:
    Derived() { cout << "D"; }
    ~Derived() { cout << "d"; }
};
int main() { Derived obj; }
```

A. BDdb B. DBbd C. BDdb D. DBdb

Answer: C. Why: Construction order is Base then Derived ("BD"); destruction is the exact reverse, Derived then Base ("db") — giving "BDdb".

Q2. [Trap] What is the key difference between the Ambiguity Problem and the Diamond Problem in multiple inheritance?

A. They are the same thing B. Ambiguity is a name conflict between unrelated base classes (fixed with `::`); Diamond is duplicate copies of a shared ancestor (fixed with virtual inheritance) C. Diamond is fixed with `::`; Ambiguity with virtual inheritance D. Neither can be fixed in C++

Answer: B. Why: These are distinct problems with distinct fixes — conflating them (or swapping their fixes) is the classic exam trap.

Q3. [Code trace] What does this print?

```

class Animal {
public:
    virtual void sound() { cout << "Generic sound"; }
};
class Cat : public Animal {
public:
    void sound() override { cout << "Meow"; }
};
int main() {
    Animal *ptr = new Cat();
    ptr->sound();
}

```

A. Generic sound B. Meow C. Compile error D. Undefined behavior

Answer: B. Why: `sound()` is virtual, so calling it through a base pointer dispatches to the actual object's (Cat's) override at runtime — the definition of runtime polymorphism.

Q4. [Trap] Why is a virtual destructor essential in a polymorphic base class?

A. It makes the class faster B. Without it, deleting a derived object through a base pointer only calls the base destructor, leaking derived-class resources C. It is required by the C++ standard for all classes D. It prevents object slicing

Answer: B. Why: `Base *ptr = new Derived(); delete ptr;` with a non-virtual `~Base()` only runs `~Base()`, skipping `~Derived()` and any cleanup it performs.

Q5. A class containing at least one pure virtual function is called:

A. A concrete class B. A friend class C. An Abstract Class D. A static class

Answer: C. Why: By definition, ≥ 1 pure virtual function (`= 0`) makes a class abstract — it cannot be instantiated until a derived class implements all such functions.

Q6. [Trap] What happens if a derived class of an abstract base implements only SOME (not all) of the inherited pure virtual functions?

A. It becomes concrete and can be instantiated B. It remains abstract and still cannot be instantiated C. Compile error immediately D. The unimplemented functions default to no-ops

Answer: B. Why: ALL pure virtual functions must be overridden for a derived class to become concrete; partial implementation leaves it abstract too.

Q7. Object Slicing occurs when:

A. A derived object is accessed via a base-class pointer B. A derived-class object is assigned BY VALUE to a base-class object, losing the derived-only members C. A base class has a virtual destructor D. Multiple inheritance is used

Answer: B. Why: Value assignment only copies the base-class portion of the object; the derived part is "sliced off" — polymorphism is also broken in the sliced copy.

Q8. [Code trace] Given:

```

class Base { public: Base(int x) { cout << x; } };
class Derived : public Base {
public:
    Derived() { }
};

```

What happens when you try `Derived d;` ?

A. Compiles fine, prints nothing B. Compile error — no matching `Base()` constructor C. Compiles, prints 0 D. Runtime crash

Answer: B. Why: `Base` has no default constructor, and `Derived`'s constructor doesn't forward any argument to it — the compiler cannot find a `Base()` to call implicitly.

Q9. Function Overriding requires all of the following EXCEPT:

A. Inheritance relationship B. Same function name and parameter list C. The base function must be declared virtual D. The function must be static

Answer: D. Why: Overriding needs a virtual (non-static) base function; a static function cannot be virtual and cannot be overridden in the polymorphic sense.

Q10. Virtual Inheritance is specifically used to solve:

A. The Ambiguity Problem B. The Diamond Problem — ensuring only one shared copy of a common base C. Object slicing D. Function overloading conflicts

Answer: B. Why: `virtual public Base` in both intermediate classes ensures the final derived class contains only one `Base` subobject instead of two duplicate copies.

Q11. Which STL component is responsible for traversing a container like `vector` or `list` ?

A. Algorithm B. Iterator C. Template D. Functor

Answer: B. Why: Iterators are STL's mechanism for moving through container elements; Containers store data and Algorithms (like `sort`) operate on it via iterators.

Q12. [Trap] Compile-time polymorphism differs from runtime polymorphism in that it:

A. Requires at least one virtual function B. Is resolved by the compiler before the program runs, via overloading — no virtual function needed C. Is always slower due to dynamic dispatch D. Only applies to operators, never functions

Answer: B. Why: Compile-time (static) polymorphism = function/operator overloading, resolved purely by signature matching at compile time; runtime polymorphism specifically needs virtual functions and dynamic dispatch.

9. Data Structures Basics & Arrays

9.1 Classification of Data Structures

- **Primitive:** system-defined, directly operated on by machine instructions — int, float, char, pointer.
- **Non-Primitive:** built from primitives — **Derived** (Arrays — system-provided but more complex) and **User-Defined** (Linked List, Stack, Queue, Tree, Graph — custom structure + operations).
- **Linear** (single level, elements have exactly one predecessor/successor, traversed in one run): Array, Stack, Queue, Linked List.
- **Non-linear** (hierarchical, multiple levels, cannot be traversed in a single run): Tree, Graph.
- **Homogeneous** (all elements same type — Array) vs **Heterogeneous** (mixed types — Struct, Union).
- **TRAP:** An array is a **derived**, not primitive, data structure — it's built on top of primitive types.

9.2 Arrays — Basics

- Contiguous memory, same data type, accessed via index in **O(1)**.
- Size fixed at declaration (C): $\text{Size} = (\text{Upper Bound} - \text{Lower Bound} + 1) * \text{element size}$.
- **Advantages:** O(1) random access, cache-friendly, simple. **Disadvantages:** fixed size, costly insertion/deletion (O(n) shifting), homogeneous only.
- Indexing conventions: 0-based (C/Java), 1-based, or n-based (arbitrary/negative lower bound — common in GATE-style questions).

9.3 1D Array Addressing

Address($a[k]$) = $B + W * (k - \text{LowerBound})$
 B = base address, W = size of each element, k = index

Worked Example

Array $a[-6..6]$, $W=4$ bytes, $\text{Base}=3500$. Find address of $a[0]$.
 Address = $3500 + 4(0 - (-6)) = 3500 + 46 = \mathbf{3524}$.

9.4 2D Array Addressing — Row-Major & Column-Major

Row-Major: Address($a[i][j]$) = $B + W * [(U2-L2+1)*(i-L1) + (j-L2)]$
 Column-Major: Address($a[i][j]$) = $B + W * [(U1-L1+1)*(j-L2) + (i-L1)]$
 $L1, U1$ = row lower/upper bound $L2, U2$ = column lower/upper bound
 $(U2-L2+1)$ = number of columns $(U1-L1+1)$ = number of rows

TRAP: Row-major uses **number of columns** as the multiplier for the row offset; column-major uses **number of rows** as the multiplier for the column offset — easy to swap under pressure.

Worked Example

Array $\text{VAL}[1..15][1..10]$, $W=4$ bytes, $\text{Base}=1500$. Find address of $\text{VAL}[12][9]$.
 Row-major: $\text{numCols}=10$; Address = $1500 + 4[10(12-1) + (9-1)] = 1500 + 4[110+8] = 1500 + 472 = 1972$.
 Column-major: $\text{numRows}=15$; Address = $1500 + 4[15(9-1) + (12-1)] = 1500 + 4[120+11] = 1500 + 524 = \mathbf{2024}$.

9.5 3D / N-Dimensional Arrays

- Generalizes row-major addressing: each dimension contributes an offset multiplied by the product of sizes of all *later* dimensions.

Address($A[i_1, i_2, \dots, i_n]$) = $B + (i_1-L1)*(U2-L2+1)*(U3-L3+1)*\dots*(U_n-L_n+1)$
 $+ (i_2-L2)*(U3-L3+1)*\dots*(U_n-L_n+1)$
 $+ \dots + (i_n-L_n) \quad [\text{all } \times W]$

9.6 Special Matrix Compact Storage (save memory by storing only non-zero region)

Lower Triangular Matrix (elements only where $\text{row} \geq \text{col}$; i.e., $i \geq j$), $n \times n$, 0-indexed, row-major

Address($A[i][j]$) = $B + W * [i*(i+1)/2 + j]$ (valid only for $j \leq i$)

Upper Triangular Matrix ($i \leq j$), $n \times n$, 0-indexed, row-major

elements before row $i = i*n - i*(i-1)/2$
 Address($A[i][j]$) = $B + W * [i*n - i*(i-1)/2 + (j - i)]$ (valid only for $j \geq i$)

Worked Examples

Lower triangular, $n=6$, $\text{Base}=1000$, $W=4$. Find address of $A[4][2]$.
 $\text{index} = 45/2 + 2 = 10+2 = 12 \rightarrow \text{Address} = 1000 + 412 = \mathbf{1048}$.

Upper triangular, $n=6$, $\text{Base}=2000$, $W=2$. Find address of $A[2][4]$.
 elements before row 2 = $26 - 21/2 = 12-1 = 11$; offset = $4-2 = 2$; index = 13 $\rightarrow \text{Address} = 2000 + 213 = 2026^*$.

9.7 Space Needed for Special $n \times n$ Matrices (classic GATE space-complexity questions)

Matrix type	Condition	Non-zero elements to store
Lower/Upper Triangular	$i \geq j$ (or $i \leq j$)	$n(n+1)/2$
Symmetric ($A[i][j] = A[j][i]$)	mirror about diagonal	$n(n+1)/2$ (only need one triangle incl. diagonal)
Tridiagonal ($A[i][j] = 0$ if $ i-j > 1$)	only diagonal + immediate neighbors	$3n - 2$
Toeplitz / diagonal-constant ($A[i][j] = A[i-1][j-1]$)	constant along each diagonal	$2n - 1$ (one value per diagonal)

- **TRAP:** A **symmetric** matrix needs the SAME space as a triangular matrix $n(n+1)/2$ — since the other half is redundant, not because it's "smaller by nature."

Questions

Q1. [Numerical] Array `a[-6..6]`, base address 3500, W=4 bytes. What is the address of `a[3]` ?

A. 3512 B. 3536 C. 3524 D. 3548

Answer: B. **Why:** Address = $3500 + 4(3 - (-6)) = 3500 + 49 = 3536$.

Q2. [Numerical] `VAL[1..15][1..10]`, Base=1500, W=4, stored row-major. What is the address of `VAL[12][9]` ?

A. 1972 B. 2024 C. 1968 D. 1980

Answer: A. **Why:** Row-major: $1500 + 4(10(12-1) + (9-1)) = 1500 + 4*118 = 1972$.

Q3. [Trap] For the same array as Q2 but stored column-major, the address of `VAL[12][9]` is:

A. 1972 B. 2024 C. 1996 D. 2000

Answer: B. **Why:** Column-major: $1500 + 4(15(9-1) + (12-1)) = 1500 + 4*131 = 2024$ — a different value from row-major for the same element, since the traversal order differs.

Q4. How much storage (in terms of n) does an $n \times n$ tridiagonal matrix need, storing only non-zero elements?

A. n^2 B. $n(n+1)/2$ C. $3n - 2$ D. $2n - 1$

Answer: C. **Why:** Only the main diagonal (n) plus the super- and sub-diagonals ($n-1$ each) are non-zero: $n + (n-1) + (n-1) = 3n - 2$.

Q5. A Toeplitz matrix ($A[i][j] = A[i-1][j-1]$ for valid i, j) of size $n \times n$ requires how much storage?

A. n B. $2n - 1$ C. $n(n+1)/2$ D. n^2

Answer: B. **Why:** Every element on the same diagonal is identical, and an $n \times n$ matrix has exactly $2n - 1$ diagonals, so only $2n - 1$ distinct values need storing.

Q6. [Numerical] Lower triangular matrix, $n=8$, 0-indexed, row-major, Base=1000, W=4. Find the address of `A[5][3]`.

A. 1072 B. 1084 C. 1088 D. 1076

Answer: A. **Why:** index = $i(i+1)/2 + j = 5(5+1)/2 + 3 = 15 + 3 = 18 \rightarrow$ Address = $1000 + 4*18 = 1072$.

Q7. [Conceptual] Why does a symmetric matrix need the same storage as a triangular matrix, not less?

A. Because symmetric matrices are always smaller B. Because only one triangle (including the diagonal) determines the entire matrix — the mirrored half is redundant C. Because symmetric matrices have more non-zero entries D. There is no such relationship

Answer: B. **Why:** $A[j][i] = A[i][j]$, so storing one triangular half ($n(n+1)/2$ elements, including the diagonal) is sufficient to reconstruct the full matrix.

Q8. Which data structure below is classified as non-linear?

A. Stack B. Queue C. Tree D. Linked List

Answer: C. **Why:** Trees (and Graphs) have hierarchical, multi-level relationships between elements — unlike linear structures where each element has a single predecessor/successor.

10. Stacks — Expression Evaluation & Recursion Applications

10.1 Stack Basics

- **LIFO** (Last In First Out). Operations: **push**, **pop**, **peek/top**, **isEmpty**, **isFull** — all $O(1)$.
- **Static (array) implementation**: fixed capacity, simple, risk of overflow. **Dynamic (linked-list) implementation**: flexible size, extra pointer memory per node.
- Overflow: push when $TOP == N-1$ (array full). Underflow: pop when $TOP == -1$ (empty).

10.2 Valid Stack Permutations (Pop Sequences)

- Given a fixed **push order**, not every possible ordering of pop outputs is achievable — simulate mentally with a stack to check.

Worked Example

Push order 1,2,3,4,5. Which of these is **NOT** a valid pop sequence?

A. 3,5,4,2,1 B. 2,4,3,5,1 C. 4,3,5,2,1 D. 5,4,3,1,2

Answer: D. Trace: push 1,2,3,4,5 → pop 5,4,3 leaves stack [1,2] (2 on top). To output "1" next, 2 would have to be popped first (it's on top) — but the target wants 1 *before* 2, which is impossible once the stack is [1,2]. Options A, B, C are all achievable by direct simulation.

10.3 Expression Notations

Notation	Form	Example (a+b)
Infix	operator between operands	a + b
Prefix (Polish)	operator before operands	+ a b
Postfix (Reverse Polish)	operator after operands	a b +

- Postfix/Prefix need **no parentheses or precedence rules** — directly stack-evaluable, which is why compilers/ALUs prefer them internally.

Infix → Postfix Conversion Algorithm (operator stack)

```

Scan left to right:
Operand      -> append to output
'('          -> push
')'          -> pop and append until '(' is popped (discard both parens)
Operator op  -> while stack top is an operator with GREATER OR EQUAL precedence
               (respecting left-associativity), pop and append it;
               then push op
End: pop all remaining operators to output

```

Worked Example — Convert $A + B * C - D$ to postfix

Scan: $A \rightarrow \text{out}(A)$; $+\rightarrow \text{push}(+)$; $B \rightarrow \text{out}(A B)$; $\rightarrow \text{push}()$ [since + has lower precedence, don't pop] (stack: +);

$C \rightarrow \text{out}(A B C)$; $\rightarrow \text{pop} * (\text{out: } A B C)$, pop + (out: $A B C * +$) [both $\geq$ precedence of -, push - (stack: -)];

End $\rightarrow$ pop - (out: $A B C * + D$ - ... wait D must be appended first)

Corrected scan: $A(\text{out:}A) + B(\text{out:}AB) (\text{out:}ABC \text{ after } C) - (\text{out:}ABC +) D(\text{out:}ABC + D) \rightarrow \text{pop remaining} - \rightarrow \text{Postfix} = A B C * + D -$

Postfix Evaluation Algorithm

```

Scan left to right: operand -> push; operator -> pop 2 (top=second operand), apply, push result.

```

Worked Example — Evaluate $6\ 2\ 3\ *\ +\ 5\ -$

```

push 6; push 2; push 3 → stack[6,2,3]
* → pop 3,2 → 2*3=6 → push 6 → stack[6,6]
+ → pop 6,6 → 6+6=12 → push 12 → stack[12]
push 5 → stack[12,5]
- → pop 5,12 → 12-5=7 → push 7 → stack[7]
Result = 7*

```

10.4 Recursion & the Call Stack

- Recursion needs a **base case** (stops recursion) + a **recursive case**. Each call gets its own **stack frame** (activation record) with its own copies of parameters/locals — this is *why* recursion has memory overhead and can **stack overflow** with excessive depth.
- **Direct** recursion (A calls A) vs **Indirect** recursion (A calls B calls A).
- **Tail recursion**: the recursive call is the LAST operation (some compilers optimize this into a loop). **Head/non-tail recursion**: work happens AFTER the recursive call returns (e.g., printing after the recursive call) — this is why reversing print-order (print-before-call vs print-after-call) is a classic trace question (see recall pattern below).
- **Recognition pattern**: print-BEFORE-recursive-call → prints in original order; print-AFTER-recursive-call → prints in REVERSED order (since printing happens during the unwind phase).

Worked Trace

```

void fun(int x) { if (x>0) { fun(x-1); printf("%d",x); } }
fun(4) call chain: fun(4)->fun(3)->fun(2)->fun(1)->fun(0)[base, x=0 not>0, returns]
Unwind (prints happen AFTER the recursive call, so in REVERSE call order): 1 2 3 4
Output: 1234

```

10.5 Tower of Hanoi (classic stack/recursion application)

$T(n) = 2 \cdot T(n-1) + 1$, $T(0) = 0 \Rightarrow T(n) = 2^n - 1$ (minimum moves for n disks)
 Total recursive function calls = $2^{n+1} - 1$
 Number of invocations before the FIRST disk moves = n

- Rules: move 1 disk at a time; never place a larger disk on a smaller one.

10.6 Fibonacci & Ackermann (brief)

- Fibonacci recurrence: $f(n) = f(n-1) + f(n-2)$, $f(0)=0$, $f(1)=1$ — naive recursive computation is exponential time (many repeated subcalls) since it doesn't cache results; number of invocations to compute $f(n) = 2 \cdot f(n+1) - 1$.
- Ackermann function $A(m,n)$: a classic example of a **total computable function that is NOT primitive recursive** — grows dramatically faster than any exponential/tower function (e.g., $A(4,2)$ is already astronomically large). [VERIFY exact growth values if needed for a specific question — the key exam fact is just "not primitive recursive, grows faster than any primitive recursive function."]

Questions

Q1. [Trap] Push order is 1,2,3,4,5. Which of the following is NOT achievable as a pop sequence?

A. 3,5,4,2,1 B. 2,4,3,5,1 C. 4,3,5,2,1 D. 5,4,3,1,2

Answer: D. **Why:** After popping 5,4,3, the stack holds [1,2] with 2 on top — 1 cannot be popped before 2, so "...1,2" at the end is impossible.

Q2. [Numerical] Convert infix $A + B * C - D$ to postfix.

A. $A B C * + D -$ B. $A B + C * D -$ C. $A B C + * D -$ D. $A B * C + D -$

Answer: A. **Why:** $*$ binds tighter than $+$ / $-$; BC is grouped first ($\rightarrow BC$), then added to A ($\rightarrow ABC+$), then D is subtracted ($\rightarrow ABC+D-$).

Q3. [Numerical] Evaluate postfix expression $6 \ 2 \ 3 \ * \ + \ 5 \ -$.

A. 5 B. 7 C. 13 D. 19

Answer: B. **Why:** $2*3=6$; $6+5=11$; $11-6=5$.

Q4. [Numerical] Tower of Hanoi with 5 disks — minimum number of moves required?

A. 10 B. 25 C. 31 D. 32

Answer: C. **Why:** $T(n) = 2^n - 1 = 2^5 - 1 = 31$.

Q5. [Trap] `void fun(int x){ if(x>0){ printf("%d",x); fun(x-1); } }` called as `fun(3)`. What is printed?

A. 123 B. 321 C. 3210 D. 0123

Answer: B. **Why:** Print happens BEFORE the recursive call, so output follows call order directly: 3, then 2, then 1 $\rightarrow$ "321".

Q6. [Trap] `void fun(int x){ if(x>0){ fun(x-1); printf("%d",x); } }` called as `fun(3)`. What is printed?

A. 123 B. 321 C. 3210 D. 0123

Answer: A. **Why:** Print happens AFTER the recursive call (during unwind), so output is in reverse call order: 1, 2, 3 $\rightarrow$ "123".

Q7. [Numerical] `int fun(int x,int y){ if(y==0) return 0; return x+fun(x,y-1); }` — what does `fun(x,y)` compute in general, and what is `fun(4,3)`?

A. $x+y = 7$ B. $xy = 12$ C. $x^y = 64$ D. $y-x = -1$

Answer: B. **Why:** The function adds x to itself y times ($x+x+x+\dots+0$), i.e., computes xy ; $\text{fun}(4,3) = 4+4+4+0 = 12$.

Q8. A recursive function's recursive call is the very last statement executed in the function body. This is called:

A. Head recursion B. Tail recursion C. Indirect recursion D. Mutual recursion

Answer: B. **Why:** Tail recursion places the recursive call last, with no pending work after it returns — some compilers can optimize this into an iterative loop.

Q9. Why can excessive (or infinite) recursion cause a program crash?

A. The CPU cache overflows B. Each recursive call consumes stack frame memory for its own parameters/locals, and unbounded calls exhaust the call stack (stack overflow) C. Recursive functions cannot use local variables D. It causes a compile-time error

Answer: B. **Why:** Every call pushes a new activation record onto the call stack; without a reachable base case, this grows until the stack's memory limit is exceeded.

Q10. [Numerical] `int fun(int x,int y){ if(x==0) return y; return fun(x-1, x+y); }` — evaluate `fun(4,3)`.

A. 7 B. 10 C. 13 D. 12

Answer: C. **Why:** $\text{fun}(4,3) \rightarrow \text{fun}(3,7) \rightarrow \text{fun}(2,10) \rightarrow \text{fun}(1,12) \rightarrow \text{fun}(0,13) \rightarrow$ returns 13.

11. Queues & Linked Lists

11.1 Queue Basics

- **FIFO** (First In First Out): insert (**enqueue**) at REAR, remove (**dequeue**) at FRONT — both **O(1)**.
- Naive linear-array queue: $REAR = REAR + 1$ on enqueue, $FRONT = FRONT + 1$ on dequeue — but this **wastes space at the front** after repeated dequeues (front slots become unusable), motivating the Circular Queue.
- Applications: shared-resource scheduling, asynchronous data transfer between processes (producer/consumer buffering), load balancing, printer spooling.

11.2 Circular Queue

- Wraps front/rear via modulo: $R = (R+1) \% N$ on enqueue, $F = (F+1) \% N$ on dequeue.
- **Overflow condition**: $(F==0 \ \&\& \ R==N-1) \text{ OR } (F==R+1)$ — the second clause detects the wrapped-around-full case.
- **Underflow condition**: $F == -1$.
- **TRAP**: Distinguishing **full** from **empty** is subtle — a plain $F==R$ could mean either "one element left" or "empty," so implementations use a sentinel ($F=R=-1$ for empty), an explicit counter, or sacrifice one slot to disambiguate.

11.3 Priority Queue

- Each element has a **priority**; dequeue always removes the **highest-priority** element (not strict arrival order — ties broken by arrival order/FIFO among equal priorities).
- Typically implemented via a **Heap** for $O(\log n)$ insert/extract (see Trees chapter) — a plain sorted array/list would need $O(n)$ insertion or $O(n)$ extraction.
- Reversing a queue using only a stack: dequeue everything into a stack (reverses order), then dequeue-from-stack (pop) back into the queue — restores original FIFO order after this double-reversal... **TRAP**: actually dequeuing into a stack and pushing each popped item back into the queue via enqueue **reverses the queue's order once** (net single reversal, not restoration) — verify the exact intended behavior against the specific pseudocode given, since "push into stack then push back into queue" is a classic "reverse a queue using one stack" utility.

11.4 Linked List Basics

- Node = {data, next pointer} . **head** pointer references the first node; last node's `next = NULL` .
- **vs Array**: dynamic size (no fixed pre-allocation), $O(1)$ insert/delete GIVEN a pointer to the location — but **no random access** ($O(n)$ to reach the i -th node), and extra memory overhead per node for the pointer.
- **TRAP**: Linked lists give up **cache locality/random access** (array's strength) in exchange for dynamic sizing and cheap insert/delete.

11.5 Linked List Variants

Variant	Key property
Singly Linked List (SLL)	one <code>next</code> pointer; forward traversal only
Circular SLL	last node's <code>next</code> points back to the first node (no NULL) — needs an explicit stopping condition (e.g., "stop when back at start") to avoid infinite traversal
Header Linked List	a dummy header node (no real data) always present, even when list is "empty" — simplifies insert/delete at the head (no special-casing an empty list) and can store metadata (e.g., length)
Header Circular LL	header node + circular structure combined
Doubly Linked List (DLL)	each node has both <code>next</code> and <code>prev</code> — enables $O(1)$ backward traversal and easier deletion (no need to track the previous node separately), at the cost of extra memory per node
Header Circular Doubly LL	header + circular + bidirectional — most flexible, most memory overhead

11.6 Core Operations (verified logic)

- **Insert at head**: `new->next = *head; *head = new;` — $O(1)$.
- **Delete at head**: `temp = *head; *head = (*head)->next; free(temp);` — $O(1)$.
- **Iterative reversal**: walk with three pointers (`prev=NULL`, `curr=head`, `next`), at each step save `next=curr->next`, reverse the link `curr->next=prev`, then advance `prev=curr; curr=next`. At the end, ***head_ref = prev** (prev ends up as the new head, i.e. the former last node) — **TRAP**: a common "fill in the blank" question asks for exactly this missing final statement.
- **Recursive reversal**: `reverse(cn, pn) : if cn==NULL return pn; else save nn=cn->next; set cn->next=pn; recurse reverse(nn, cn)`.

11.7 Comparison Table

Feature	Array	Singly LL	Doubly LL	Circular LL
Random access	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Insert/delete at front	$O(n)$ (shift)	$O(1)$	$O(1)$	$O(1)$
Insert/delete at end (no tail ptr)	$O(1)$ (amortized)	$O(n)$	$O(n)$ ($O(1)$ with tail ptr)	$O(n)$
Backward traversal	$O(1)$ (indexable)	Not possible	$O(1)$	Not possible (SLL variant)
Extra memory/node	None	1 pointer	2 pointers	1 pointer

Questions

Q1. [Trap] Distinguishing a "full" circular queue from an "empty" one using only front/rear pointers is tricky because:

A. Circular queues cannot become full B. `front == rear` can represent BOTH the empty and the "one element left" states, requiring an extra counter or sentinel to disambiguate C. Modulo arithmetic is undefined for queues D. There is no such ambiguity

Answer: B. **Why:** Without an explicit counter or sacrificed slot, `front==rear` is structurally ambiguous between "0 elements" and "N elements" in some implementations — careful sentinel handling (like `F=R=-1` for empty) is required.

Q2. Why is a naive (non-circular) array-based queue considered wasteful over time?

A. It cannot support enqueue B. Dequeued front slots become permanently unusable, wasting array space even though logically free C. It requires $O(n)$ enqueue D. It cannot store more than 2 elements

Answer: B. **Why:** As FRONT increments with each dequeue, the slots before it are never reused in a simple linear array queue — motivating the circular (modulo-wraparound) design.

Q3. A Priority Queue is most efficiently implemented using which data structure for $O(\log n)$ insert/extract?

A. Sorted array B. Unsorted linked list C. Heap D. Stack

Answer: C. **Why:** A heap supports $O(\log n)$ insertion and $O(\log n)$ extraction of the max/min element, unlike a sorted array ($O(n)$ insert) or unsorted list ($O(n)$ extraction).

Q4. In a Doubly Linked List, what is the main cost paid for gaining backward traversal capability?

A. $O(n)$ random access (same as SLL, no change) B. An extra pointer (`prev`) per node, increasing memory overhead C. Loss of the ability to insert at the head D. It becomes non-linear

Answer: B. **Why:** Each DLL node stores both `next` and `prev`, doubling the pointer overhead compared to a Singly Linked List.

Q5. [Trap] What is the key advantage of a Header Linked List over a plain Singly Linked List?

A. It uses less memory overall B. It eliminates special-case handling for insertion/deletion at an empty list or at the head, since a dummy header node is always present C. It allows $O(1)$ random access D. It removes the need for a next pointer

Answer: B. **Why:** Because the header node always exists (even for a logically empty list), insert/delete code doesn't need special "is the list empty?" branches.

Q6. [Numerical/Trace] `reverse(struct node** head_ref)` iteratively swaps `next` pointers using `prev`, `current`, `next` as described in the notes. What single statement correctly completes the function after the loop?

A. `*head_ref = current;` B. `*head_ref = next;` C. `*head_ref = prev;` D. `*head_ref = NULL;`

Answer: C. **Why:** After the loop, `current` is NULL and `prev` holds the former last node — now the new head — so the head pointer must be updated to `prev`.

Q7. [Trace] For linked list `1->2->3->4->5->6`, what does this print? `void fun(node* s){ if(s==NULL) return; printf("%d ",s->data); if(s->next!=NULL) fun(s->next->next); printf("%d ",s->data); }` called with the head.

A. 1 4 6 6 4 1 B. 1 3 5 1 3 5 C. 1 2 3 5 D. 1 3 5 5 3 1

Answer: D. **Why:** `fun(1)` prints 1, calls `fun(3)`; `fun(3)` prints 3, calls `fun(5)`; `fun(5)` prints 5, calls `fun(NULL)` [node 6's `next->next`, no-op]; unwinding prints 5, 3, 1 again → "1 3 5 5 3 1".

Q8. [Trap] `void traverse(Node *head){ while(head->next!=NULL){ printf("%d",head->data); head=head->next; } }` Which statement about this function is FALSE?

A. It may crash if the list is empty B. It fails to print the last node's data even when the list is non-empty C. It is "incorrect" specifically because it reassigns the local `head` parameter D. None of the above

Answer: C. **Why:** Reassigning the local copy of `head` (pass-by-value) has no effect on the caller's pointer and causes no bug — so labeling this as the cause of incorrectness is a false claim. (A and B are both true statements about the function's real behavior.)

Q9. Which linked list variant requires an explicit stopping condition (rather than a NULL check) to avoid an infinite traversal loop?

A. Singly Linked List B. Doubly Linked List C. Circular Linked List D. None — all use NULL termination

Answer: C. **Why:** A circular list's last node points back to the first node instead of NULL, so traversal must track the starting node or use a counter to know when to stop.

12. Trees — BST, AVL, Heaps & K-ary Trees

12.1 Tree Terminology & Binary Tree Formulas

- Root, leaf (no children), internal node (≥ 1 child), degree (# children), level/depth (root=level 0 or 1 depending on convention — state which convention a question uses), edges = $N-1$ for N nodes.

Max nodes at level L (root=level 0) = 2^L
 Max nodes in a tree of height h = $2^{(h+1)} - 1$
 Min height for n nodes (i.e. most balanced) = $\text{ceil}(\log_2(n+1)) - 1$ [0-indexed height]

12.2 Traversals

Traversal	Order	Note
Preorder	Root, Left, Right	used to serialize/clone a tree
Inorder	Left, Root, Right	produces SORTED order for a BST — key fact
Postorder	Left, Right, Root	used to safely delete a tree (children before parent)
Level-order	top-to-bottom, left-to-right per level	BFS-style, uses a queue

- Common to all three DFS traversals: **left subtree is always visited before right subtree** — only the ROOT's position changes.

Worked Example

```

      4
     / \
    2   6
   / \ / \
  1  3 5 7
Preorder: 4 2 1 3 6 5 7
Inorder:  1 2 3 4 5 6 7 (sorted – this is a BST)
Postorder: 1 3 2 5 7 6 4
Level-order: 4 2 6 1 3 5 7
  
```

12.3 Reconstructing a Tree from Traversals

- Inorder + Preorder** → unique tree (root = first of preorder, split inorder around it).
- Inorder + Postorder** → unique tree (root = last of postorder).
- TRAP: Preorder + Postorder alone is NOT sufficient** in general (only works if the tree is guaranteed full, i.e., every node has 0 or 2 children) — Inorder is essential to locate the split point otherwise.

Worked Example (reconstruct from Preorder=4 2 1 3 6 5 7, Inorder=1 2 3 4 5 6 7)

Root = 4 (first of preorder). In inorder, 4 splits into left={1,2,3}, right={5,6,7}.
 Left preorder(next 3) = 2,1,3 → root=2, splits into left={1}, right={3}.
 Right preorder(next 3) = 6,5,7 → root=6, splits into left={5}, right={7}.
 Reconstructs exactly the tree shown above. ✓

12.4 Binary Search Tree (BST)

- Left subtree < node < right subtree (both subtrees also BSTs).
- Insertion:** traverse like a search, insert at the NULL spot found. **Deletion** 3 cases: leaf (just remove), one child (splice child up to parent), two children (replace with **inorder predecessor** [max of left subtree] or **inorder successor** [min of right subtree], then delete that node from its original position).
- TRAP:** the inorder successor used in two-child deletion is always either a **leaf, or a node with an empty LEFT child** (it's the leftmost node of the right subtree, so it can only have a right child, never a left one).

Operation	Average	Worst case
Search / Insert / Delete	$O(\log n)$	$O(n)$ (degenerates to a linked list if inserted in sorted order)

- Number of distinct BSTs possible with n distinct keys = Catalan number $C_n = \frac{(2n)!}{[(n+1)! n!]}$ → $C_3=5$, $C_4=14$ (both classic GATE numbers, verify by direct formula).**
- Expected/average comparisons for a random search** = (sum over all nodes of [node's level] $\times$ 1) / (number of nodes), where root is level 1.

Worked Example — Expected comparisons

```

Tree:
      10 (L1)
     /  \
    5    20 (L2)
   /  \  /  \
  4   15 30 (L3)
   /
  11 (L4)
Levels: 10=1, 5=2, 20=2, 4=3, 15=3, 30=3, 11=4 (7 nodes)
Expected comparisons = (1*1 + 2*2 + 3*3 + 4*1) / 7 = (1+4+9+4)/7 = 18/7 ≈ 2.57
  
```

12.5 AVL Tree (self-balancing BST)

- **Balance Factor(N) = height(left) – height(right)**, must be in **{-1, 0, +1}** for every node.
- Guarantees **O(log n) worst case** for search/insert/delete (unlike plain BST's O(n) worst case) — this IS the entire point of AVL.

Rotation needed | Fix
 LL (left-left) -> single Right rotation
 RR (right-right)-> single Left rotation
 LR (left-right) -> Left rotation then Right rotation (double)
 RL (right-left) -> Right rotation then Left rotation (double)

- After any single insertion, **at most 2 rotations** are ever needed to restore balance.

Structure	Search/Insert/Delete (avg)	Worst case
Plain BST	$O(\log n)$	$O(n)$
AVL Tree	$O(\log n)$	$O(\log n)$

12.6 Complete Binary Tree & Heap

- **Complete Binary Tree:** all levels full except possibly the last, which fills **strictly left to right** — this property is exactly what allows array-based storage without wasted gaps.
- For a node at (0-indexed) array position **i**: **parent(i) = (i-1)/2** (integer division), **left(i) = 2i+1**, **right(i) = 2i+2**.
- **Max-Heap:** parent $\geq$ both children. **Min-Heap:** parent $\leq$ both children. A heap is ALWAYS a complete binary tree, but NOT necessarily a BST (no left<root<right ordering across subtrees, only parent-child ordering).
- **TRAP:** A heap guarantees order only between parent and child, NOT between siblings or across subtrees — don't assume heap order = sorted order.

Worked Example — Valid Max-Heap Check (GATE 2006 style)

Array: 23,17,14,7,13,10,1,5,6,12 (0-indexed). Check every parent(i) $\geq$ child:

idx1=17 $\leq$ 23 $\checkmark$, idx2=14 $\leq$ 23 $\checkmark$, idx3=7 $\leq$ 17 $\checkmark$, idx4=13 $\leq$ 17 $\checkmark$, idx5=10 $\leq$ 14 $\checkmark$, idx6=1 $\leq$ 14 $\checkmark$, idx7=5 $\leq$ 7 $\checkmark$, idx8=6 $\leq$ 7 $\checkmark$, idx9=12 $\leq$ 13 $\checkmark$ → **Valid max-heap.**

Worked Example — Two Delete-Max Operations

Starting heap array: 25,14,16,13,10,8,12.

Delete 1: remove 25; move last (12) to root → [12,14,16,13,10,8]; sift down: 12<16(larger child)→swap→ [16,14,12,13,10,8]; 12 vs child 8 → 12 $\geq$ 8, stop. Result: 16,14,12,13,10,8.

Delete 2: remove 16; move last (8) to root → [8,14,12,13,10]; sift down: 8<14→swap→ [14,8,12,13,10]; 8 vs children 13,10 → 8<13(larger)→swap→ [14,13,12,8,10]; no more children. Result: **14,13,12,8,10**.

12.7 Threaded Binary Tree

- Replaces NULL pointers with **threads** to the inorder predecessor/successor — enables traversal **without recursion or an explicit stack**. Single-threaded (only left OR right NULLs replaced) vs Double-threaded (both).

12.8 K-ary Trees (Full / Strict k-ary tree: every node has 0 or exactly k children)

Leaf nodes $L = I \cdot (k-1) + 1$
 Total nodes $N = I + L = I \cdot k + 1$

(I = internal nodes). **Full** k-ary (node-based: 0 or k children) is distinct from **Complete** k-ary (level-based: all levels full except possibly last, filled left-to-right).

Worked Examples (all independently verified)

Full 3-ary tree with 41 leaves → find total nodes: $41 = I \cdot 2 + 1 \rightarrow I = 20 \rightarrow N = 20 + 41 = 61$.

Full 4-ary tree with 85 total nodes → find internal nodes: $85 = I \cdot 4 + 1 \rightarrow I = 21$.

Full 5-ary tree with 26 internal nodes → find leaves: $L = 26 \cdot 4 + 1 = 105$.

Full k-ary tree, 121 total nodes, 30 internal nodes → find k: $121 = 30k + 1 \rightarrow k = 4$.

Full k-ary tree, 67 leaves, 33 internal nodes → find k: $67 = 33(k-1) + 1 \rightarrow k-1 = 2 \rightarrow k = 3$.

Questions

Q1. Which traversal of a Binary Search Tree always produces output in sorted order?

A. Preorder B. Inorder C. Postorder D. Level-order

Answer: B. Why: Inorder visits Left-Root-Right, which for a BST always yields ascending sorted order.

Q2. [Trap] Which pair of traversals is generally NOT sufficient to uniquely reconstruct a binary tree?

A. Preorder and Inorder B. Inorder and Postorder C. Preorder and Postorder D. Level-order and Inorder

Answer: C. Why: Without Inorder, Preorder+Postorder cannot locate the split between left/right subtrees uniquely (unless the tree is known to be full).

Q3. [Numerical] How many structurally distinct BSTs can be formed using 4 distinct keys?

A. 4 B. 14 C. 24 D. 42

Answer: B. Why: Catalan number $C_4 = (2 \times 4)! / ((5)! \times 4!) = 14$.

Q4. What is the worst-case time complexity of search in a plain (unbalanced) BST?

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

Answer: C. Why: Inserting sorted data degenerates a BST into a linked-list shape, making search $O(n)$ in the worst case — this is exactly what AVL trees fix.

Q5. In an AVL tree, a node's Balance Factor is defined as:

A. height(right) – height(left) B. height(left) – height(right) C. height(left) + height(right) D. depth(node) – height(node)

Answer: B. Why: $BF(N) = \text{height(LeftSubtree)} - \text{height(RightSubtree)}$; a valid AVL node must have $BF \in \{-1, 0, +1\}$.

Q6. An imbalance classified as "RL" (right-left) is fixed by:

A. A single left rotation B. A single right rotation C. Right rotation then left rotation D. Left rotation then right rotation

Answer: C. Why: RL imbalance requires a right rotation on the right child first, then a left rotation on the node itself (double rotation).

Q7. [Numerical] For the 0-indexed array `23, 17, 14, 7, 13, 10, 1, 5, 6, 12`, which of these correctly identifies it as:

A. An invalid max-heap because $\text{idx}3(7) < \text{idx}7(5)$'s parent check fails B. A valid max-heap — every parent is $\geq$ both its children C. A valid min-heap D. Not a complete binary tree

Answer: B. **Why:** Checking every parent-child pair ($\text{idx}0=23$ down to $\text{idx}9=12$) confirms parent $\geq$ child throughout — a valid max-heap.

Q8. [Numerical] Starting from max-heap array `25, 14, 16, 13, 10, 8, 12`, what is the array after 2 delete-max operations?

A. 14, 13, 12, 10, 8 B. 14, 12, 13, 8, 10 C. 14, 13, 8, 12, 10 D. 14, 13, 12, 8, 10

Answer: D. **Why:** First delete-max: 25 removed, 12 moved to root, sifts down to give `16, 14, 12, 13, 10, 8`. Second delete-max: 16 removed, 8 moved to root, sifts down to give `14, 13, 12, 8, 10`.

Q9. [Numerical] A full 3-ary tree has 41 leaf nodes. How many total nodes does it have?

A. 20 B. 41 C. 60 D. 61

Answer: D. **Why:** $L = I(k-1) + 1 \rightarrow 41 = 2I + 1 \rightarrow I = 20$; Total = $I + L = 20 + 41 = 61$.

Q10. [Numerical] A full k-ary tree has 121 total nodes and 30 internal nodes. What is k?

A. 3 B. 4 C. 5 D. 6

Answer: B. **Why:** $N = I(k+1) \rightarrow 121 = 30k + 1 \rightarrow k = 4$.

Q11. [Trap] For a complete binary tree stored in a 0-indexed array, the parent of the node at index i is at index:

A. $i/2$ B. $(i-1)/2$ (integer division) C. $2i$ D. $(i+1)/2$

Answer: B. **Why:** With children at $2i+1$ and $2i+2$, solving for parent gives $(i-1)/2$ — a common off-by-one trap if you assume 1-indexing instead.

Q12. A Threaded Binary Tree replaces NULL pointers with threads mainly to:

A. Save memory by removing all pointers B. Enable traversal without recursion or an explicit stack C. Convert the tree into a BST D. Allow more than 2 children per node

Answer: B. **Why:** Threads point to the inorder predecessor/successor, letting traversal algorithms walk the tree iteratively without needing a call stack.

13. Graphs — Representation, DFS, BFS & Topological Sort

13.1 Graph Terminology & Representation

- **Graph** = Vertices (nodes) + Edges (ordered pairs (u,v) for directed, unordered for undirected). Weighted graph: edges carry a cost/value.
- **Degree** (undirected) = number of incident edges. Directed graph: **in-degree** (edges pointing in) and **out-degree** (edges pointing out), tracked separately.
- **Adjacency Matrix vs Adjacency List:**

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$ — same regardless of edge count	$O(V+E)$ — efficient for sparse graphs
Edge lookup (u,v)?	$O(1)$	$O(\text{degree}(u))$
Add vertex	$O(V^2)$ (resize)	$O(1)$
Best for	Dense graphs, frequent edge queries	Sparse graphs (most real-world graphs)

- **TRAP:** Adjacency Matrix wastes $O(V^2)$ space even on a sparse graph (few edges) — Adjacency List is the standard default choice since most graphs encountered in practice ARE sparse.
- Undirected graph's adjacency matrix is always **symmetric**.

13.2 DFS (Depth-First Search)

- Uses a **stack** (explicit, or implicitly via recursion). Goes as deep as possible along one branch before backtracking.
- Needs a **visited[] array** to avoid infinite loops on cycles (unlike tree traversal).

```
DFS(v):
    visited[v] = true
    for each x adjacent to v:
        if not visited[x]: DFS(x)
```

- **Time complexity:** $O(V+E)$ with adjacency list, $O(V^2)$ with adjacency matrix.
- Applications: cycle detection, topological sort, connected components, maze/path-finding.
- **Edge classification in directed-graph DFS** (based on discovery/finish times):

Edge type	Meaning
Tree edge	Leads to a not-yet-visited vertex — forms the DFS tree
Back edge	Points to an ancestor in the DFS tree — indicates a cycle
Forward edge	Points to a descendant (already visited, not a tree edge)
Cross edge	Connects unrelated branches (neither ancestor nor descendant)

- **TRAP:** A directed graph has a cycle **if and only if** DFS finds at least one **back edge**.

13.3 BFS (Breadth-First Search)

- Uses a **queue**. Explores level by level (all neighbors of current vertex before going further).

```
BFS(v):
    visited[v] = true; enqueue(v)
    while queue not empty:
        u = dequeue()
        for each x adjacent to u:
            if not visited[x]: visited[x]=true; enqueue(x)
```

- **Time complexity:** $O(V+E)$ with adjacency list (same as DFS).
- **TRAP:** BFS finds the shortest path in an **UNWEIGHTED graph** (fewest edges) — DFS does **not** guarantee shortest path at all; it just finds *a* path.
- BFS edges (directed context) simplify to just Tree edges and Cross edges — no back/forward-edge distinction since exploration is by distance/level, not depth.

Worked Traversal Example

Graph (undirected): edges 1-2, 1-3, 2-4, 2-5, 3-6, 3-7. Start at vertex 1. Convention: visit neighbors in increasing vertex-number order.

DFS order: 1, 2, 4, 5, 3, 6, 7 (go deep via 2→4, backtrack to 2→5, backtrack to 1→3→6→7).

BFS order: 1, 2, 3, 4, 5, 6, 7 (level 0: {1}; level 1: {2,3}; level 2: {4,5,6,7}).

13.4 Topological Sort

- A linear ordering of vertices such that for every directed edge $u \rightarrow v$, **u comes before v**.
- **TRAP:** Topological sort is defined **only for a DAG (Directed Acyclic Graph)** — a graph with a cycle has **no valid topological order**.
- Two standard methods:
 1. **DFS-based:** run DFS, record each vertex's finish time; topological order = vertices in **decreasing order of finish time** (reverse of finish order).
 2. **Kahn's Algorithm (BFS-based):** repeatedly remove a vertex with **in-degree 0**, decrement its neighbors' in-degrees, repeat — order of removal is a valid topological sort. If vertices remain but none has in-degree 0, the graph has a cycle (no topological order exists).

13.5 Strongly Connected Components (SCC)

- An SCC is a **maximal set of vertices** where every vertex can reach every other vertex — meaningful **only for directed graphs**.

- Found via **Kosaraju's Algorithm** (2 DFS passes: one on the graph, one on its transpose) or **Tarjan's Algorithm** (single DFS pass using discovery times/low-link values).

Questions

Q1. Which data structure does BFS use to explore a graph, and which does DFS use?

A. Both use a stack B. BFS uses a queue, DFS uses a stack (or recursion) C. BFS uses a stack, DFS uses a queue D. Both use a priority queue

Answer: B. **Why:** BFS's queue enforces level-by-level (FIFO) exploration; DFS's stack (or the call stack via recursion) enforces depth-first (LIFO) backtracking.

Q2. [Trap] Which traversal guarantees the shortest path (fewest edges) from the source in an unweighted graph?

A. DFS B. BFS C. Both equally D. Neither

Answer: B. **Why:** BFS explores level-by-level, so the first time it reaches a vertex is guaranteed to be via the minimum number of edges; DFS may reach a vertex via a much longer path first.

Q3. A directed graph's DFS reveals at least one back edge. What can you conclude?

A. The graph is a tree B. The graph is bipartite C. The graph contains a cycle D. The graph is disconnected

Answer: C. **Why:** A back edge (pointing to an ancestor in the DFS tree) is the defining signature of a cycle in directed-graph DFS.

Q4. [Trap] Topological sorting can be performed on:

A. Any directed graph B. Only a Directed Acyclic Graph (DAG) C. Only an undirected graph D. Only a tree

Answer: B. **Why:** A cycle makes it impossible to linearly order vertices consistent with all edge directions — topological sort is undefined for cyclic graphs.

Q5. In Kahn's Algorithm for topological sort, a vertex becomes eligible for removal when:

A. Its out-degree becomes 0 B. Its in-degree becomes 0 C. It has the smallest vertex number D. All its neighbors are visited

Answer: B. **Why:** In-degree 0 means no remaining prerequisite edges point to it, so it can safely be placed next in the order.

Q6. For a SPARSE graph ($E \ll V^2$), which representation is more space-efficient?

A. Adjacency Matrix B. Adjacency List C. Both are equal D. Incidence Matrix is always best

Answer: B. **Why:** Adjacency List uses $O(V+E)$ space, scaling with actual edges; Adjacency Matrix always uses $O(V^2)$ regardless of how sparse the graph is.

Q7. Strongly Connected Components (SCCs) are a meaningful concept for:

A. Only undirected graphs B. Only directed graphs C. Both equally D. Only trees

Answer: B. **Why:** SCCs are defined by mutual reachability between vertices via directed paths — in an undirected connected graph, the whole component is trivially "strongly connected," so the concept is only interesting for directed graphs.

Q8. [Numerical/Trap] Given nodes discovered via DFS with timestamps, which edge type indicates NO cycle risk and simply connects unrelated subtrees?

A. Back edge B. Tree edge C. Cross edge D. Forward edge

Answer: C. **Why:** A cross edge connects vertices in different branches of the DFS forest (neither ancestor nor descendant) — it does not create a cycle, unlike a back edge.

Q9. Time complexity of both BFS and DFS using an adjacency list representation is:

A. $O(V^2)$ B. $O(V+E)$ C. $O(E \log V)$ D. $O(V \log V)$

Answer: B. **Why:** Both algorithms visit every vertex once and examine every edge once via the adjacency list, giving $O(V+E)$; using an adjacency matrix instead would cost $O(V^2)$.

14. Hashing

14.1 Why Hashing

- Search time comparison motivates hashing:

Structure	Search time
Unsorted array / Linked list	$O(n)$
Sorted array (binary search)	$O(\log n)$
BST	$O(n)$ worst case
AVL Tree	$O(\log n)$
Hash Table	$O(1)$ average case — independent of n

- Hash function** $H: K \rightarrow L$ maps a large key space K to a small set of table indices L .

14.2 Hash Functions

Division method: $H(K) = K \bmod m$

- Choosing table size **m as a PRIME number** reduces clustering/collisions (a common design guideline).
- Good hash function properties: easy/fast to compute, uniform distribution across slots, low collision rate, minimal clustering.

14.3 Collision Resolution

- Collision:** two different keys map to the same slot — inevitable regardless of hash function quality; must be handled.

(A) Open Addressing / Closed Hashing — all elements stored WITHIN the table itself

Technique	Formula	Key issue
Linear Probing	$h(k,i) = (h'(k) + i) \bmod m$	Primary clustering — collisions form long contiguous runs, growing worse over time
Quadratic Probing	$h(k,i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$	Secondary clustering — keys hashing to the same slot follow the same probe sequence; better than linear but not immune
Double Hashing	$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$	Best distribution among open-addressing schemes — uses a 2nd hash function for the probe step

- Load factor $\alpha = n/m$ (number of keys / table size); for open addressing α must stay < 1 (table can become full).
- Expected search/insert/delete time under open addressing $\approx 1/(1-\alpha)$.
- Deletion in open addressing is tricky: a slot cannot simply be marked empty (would break the probe chain for later searches) — must be marked **"deleted"** (tombstone): insert may reuse it, but search must continue probing past it.

(B) Chaining / Open Hashing — each slot holds a LINKED LIST

- All keys hashing to a slot are appended to that slot's list. Load factor $\alpha = n/m$ can **exceed 1**.
- TRAP on terminology (notoriously counter-intuitive): **"Open Addressing" = Closed Hashing** (no external structures, all in-table); **"Chaining" = Open Hashing** (uses external linked lists). The names are essentially swapped from what you'd guess.

Comparison

Feature	Chaining	Open Addressing
Implementation	Simpler	More complex (probing logic)
Table can fill up?	Never (list grows)	Yes — bounded by table size
Sensitivity to load factor	Lower	Higher — needs care to avoid clustering
Best when	Number of keys unknown/variable	Number/frequency of keys is known in advance
Cache performance	Worse (linked list, scattered memory)	Better (all data in one contiguous table)
Space	Some wastage (unused slots) + extra list-node memory	Slot can be used by any key regardless of its "home" — better bit-for-bit utilization

Questions

Q1. [Numerical] A hash table with 9 slots uses $h(k) = k \bmod 9$, collisions resolved by chaining. Keys inserted: 5, 28, 19, 15, 20, 33, 12, 17, 10. What are the max, min, and average chain lengths?

A. 3, 0, and 1 B. 3, 3, and 3 C. 4, 0, and 1 D. 3, 0, and 2

Answer: A. Why: Slot occupancy by $k \bmod 9$: slot1={28,19,10}{3}, slot2={20}{1}, slot3={12}{1}, slot5={5}{1}, slot6={15,33}{2}, slot8={17}{1}, slots 0,4,7 empty(0). Max=3, Min=0, Average=9 keys/9 slots=1. (This is a real GATE 2014 question.)

Q2. [Trap] "Open Addressing" in hashing terminology refers to:

A. Storing colliding keys in external linked lists B. Storing all elements within the hash table itself via probing C. A hash function that changes dynamically D. A table with no size limit

Answer: B. Why: Despite the name sounding like it might mean "open/external" storage, Open Addressing = Closed Hashing = all keys stored in-table via probing sequences — the reverse of what the name suggests.

Q3. Which open-addressing technique uses a second independent hash function to determine the probe interval?

A. Linear Probing B. Quadratic Probing C. Double Hashing D. Chaining

Answer: C. Why: Double hashing computes $h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$, using h_2 to vary the probe step per key, minimizing clustering.

Q4. Primary clustering (long runs of occupied consecutive slots) is a defining weakness of:

A. Chaining B. Linear Probing C. Double Hashing D. AVL Trees

Answer: B. **Why:** Linear probing's fixed +1 step causes collisions to merge into ever-growing contiguous blocks, degrading performance as the table fills.

Q5. Why is it recommended to choose the hash table size m as a prime number in the division method?

A. Prime numbers use less memory B. It reduces systematic clustering/collisions caused by common factors between keys and m C. It is required by the C language D. It makes chaining unnecessary

Answer: B. **Why:** If m shares factors with many keys' patterns (e.g., m is a power of 2 and keys are often even), collisions cluster into a subset of slots; a prime m avoids this systematic bias.

Q6. [Trap] Under open addressing, why can't a deleted key's slot simply be marked "empty"?

A. It would violate the hash function B. It could break the probe sequence for other keys, causing later searches to incorrectly stop early / report "not found" C. It saves memory D. Deletion is not allowed in open addressing

Answer: B. **Why:** Search relies on continuing to probe until an empty slot is hit; if a mid-sequence slot were marked truly empty after deletion, searches for keys further along that chain would wrongly terminate early. A "deleted" tombstone marker is used instead.

Q7. In chaining, unlike open addressing, the load factor $\alpha = n/m$ can be:

A. At most 0.5 B. Greater than 1 C. Never more than 1 D. Always exactly 1

Answer: B. **Why:** Since each slot holds a linked list of unbounded length, chaining never "fills up" — the number of keys n can exceed the number of slots m .

Q8. Which collision resolution technique gives the BEST cache/locality-of-reference performance, at the cost of being most prone to clustering?

A. Chaining B. Double Hashing C. Linear Probing D. Quadratic Probing

Answer: C. **Why:** Linear probing accesses sequential memory slots, maximizing cache locality, but this same sequential structure is exactly what causes primary clustering.

15. Algorithm Analysis — Asymptotic Notation & Recurrences

(High-yield in GATE/ISRO: 5-6 questions typically, mostly numerical.)

15.1 Types of Analysis

Aspect	Experimental (A Posteriori)	Asymptotic (Apriori)
When	After implementation/execution	Before implementation, purely theoretical
Measures	Actual time/space used	Estimated complexity
Depends on hardware?	Yes	No — machine-independent
Use case	Real-world comparison	Efficiency for large inputs

- **Worst-case analysis** is the most commonly used (guarantees reliability); average-case requires knowing the input distribution and is rarely tractable in practice.
- **Deterministic algorithm**: always same output/steps for a given input. **Non-deterministic**: may vary (e.g., involves randomness/choice).

15.2 Asymptotic Notations (precise definitions)

Big-O (upper bound): $O(g(n)) = \{ f(n) : 0 \leq f(n) \leq C \cdot g(n), \text{ for all } n \geq n_0 \}$
 Big-Omega (lower bound): $\Omega(g(n)) = \{ f(n) : 0 \leq C \cdot g(n) \leq f(n), \text{ for all } n \geq n_0 \}$
 Big-Theta (tight bound): $\Theta(g(n)) = \{ f(n) : C_1 \cdot g(n) \leq f(n) \leq C_2 \cdot g(n), \text{ for all } n \geq n_0 \}$
 little-o / little-omega: strict versions ($< / >$, not $\leq / \geq$) of O / Ω

- Analogy to real numbers: $O \sim "<="$, $\Omega \sim ">="$, $\Theta \sim "="$, little-o $\sim "<"$, little-omega $\sim ">"$.
- **Properties**: Reflexive ($f=O(f)$), Symmetric only for Theta ($f=\Theta(g)$ iff $g=\Theta(f)$), Transitive (all 5 notations), **Transpose symmetry**: $f=O(g)$ iff $g=\Omega(f)$.
- Sum rule: if $f=O(g)$ and $p=O(q)$, then $f+p = O(\max(g,q))$. Product rule: $f \cdot p = O(g \cdot q)$.

15.3 Complexity Class Ordering (increasing)

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

15.4 Analyzing Iterative Loops — Recognition Patterns

Loop pattern	Complexity	Why
<code>i=1; i<=n; i++</code> (step $\pm$ constant)	$O(n)$	Linear scan
<code>i=1; i<=n; i=i*c</code> (multiply by constant $c>1$)	$O(\log n)$	Halves/divides the remaining range each step
<code>i=n; i>=1; i=i/c</code> (divide by constant)	$O(\log n)$	Same — logarithmic shrink
<code>i=2; i<=n; i=i*i</code> (i squared each time)	$O(\log \log n)$	Exponent itself doubles each step
<code>i=n; i>=2; i=sqrt(i)</code>	$O(\log \log n)$	Reverse of squaring — exponent halves each step
<code>i=1; i*i<=n; i++</code> (condition on i^2)	$O(\sqrt{n})$	Loop runs while $i \leq \sqrt{n}$
<code>i=1; i<n; i=i+(n/2)</code> (step proportional to n)	$O(1)$	Step size scales with n itself — only ~ 2 -3 iterations regardless of n

- **TRAP**: A loop with step size $i = i + n/2$ looks like it depends on n but actually runs a CONSTANT number of times (~ 2 -3) since the step itself grows with n — don't assume "involves n " automatically means "not $O(1)$."
- **Nested independent loops**: total iterations = product of each loop's individual iteration count (e.g., outer $O(n) \times$ inner $O(\log n) = O(n \log n)$).

15.5 Solving Recurrences — 4 Methods

1. **Substitution**: guess the closed form, verify by mathematical induction.
2. **Recursion Tree**: draw out the recursion levels, sum work per level.
3. **Master Method**: for $T(n) = a \cdot T(n/b) + f(n)$ (see below).
4. **Change of Variable**: substitute $n = b^m$ to convert a non-standard recurrence (e.g., involving $\sqrt{n}$) into a standard one solvable by the Master Method.

Worked Recursion Examples (verified by hand)

```

rec(n): if n==1 return 1; else return rec(n-1) + n;
-> ONE recursive call -> T(n) = T(n-1) + O(1) => T(n) = O(n), Space = O(n) [recursion depth]

rec(n): if n==1 return 1; else return 2*rec(n-1) + n;
-> Still only ONE recursive call (the "2*" just scales the return value)
-> T(n) = T(n-1) + O(1) = O(n), Space = O(n)
TRAP: "2*rec(n-1)" looks like 2 calls but is actually 1 call with its result doubled – don't confuse
a multiplied return value with a second recursive invocation.

rec(n): if n==1 return 1; else return rec(n-1) + rec(n-1) + n;
-> TWO separate recursive calls, each to rec(n-1)
-> T(n) = 2*T(n-1) + O(1) => T(n) = O(2^n), Space = O(n) [max stack depth along one branch]

rec(n): if n==1 return 1; else { for(i=1;i<=n;i++) print(...); rec(n-1); }
-> T(n) = T(n-1) + O(n) => T(n) = O(n^2), Space = O(n)

rec(n): if n==1 return 1; else return 2*rec(n/2) + n;
-> ONE call (again, "2*" scales the result) -> T(n) = T(n/2) + O(n) = O(n) [Master Case 3], Space=O(log n)

rec(n): if n==1 return 1; else return rec(n/3)+rec(n/3)+rec(n/3)+n;
-> THREE calls to rec(n/3) -> T(n) = 3T(n/3) + O(n) = O(n log n) [Master Case 2], Space=O(log n)

rec(n): if n==1 return 1; else { rec(n/2); rec(n/2); for(i=1;i<=n;i++) print(...); }
-> TWO calls to rec(n/2) -> T(n) = 2T(n/2) + O(n) = O(n log n) [Master Case 2], Space=O(log n)

```

- **TRAP: Space complexity of recursion = maximum stack depth at any single point in time**, NOT the total number of calls made. Even $T(n)=2T(n-1)+O(1)$ ($O(2^n)$ time) only needs $O(n)$ space, since calls execute sequentially — only one root-to-leaf path exists on the stack at once.

15.6 Master Theorem — for $T(n) = a \cdot T(n/b) + f(n)$

Let $p = \log_b(a)$ [the "watershed" exponent]

Case 1: $f(n) = O(n^{p-\epsilon})$ for some $\epsilon > 0$ $\Rightarrow T(n) = \Theta(n^p)$ [recursive work dominates]
Case 2: $f(n) = \Theta(n^p)$ $\Rightarrow T(n) = \Theta(n^p \cdot \log n)$ [balanced – extra log factor]
Case 3: $f(n) = \Omega(n^{p+\epsilon})$, regularity holds $\Rightarrow T(n) = \Theta(f(n))$ [combine step dominates]

Worked Examples (all verified)

Recurrence	a, b	$p = \log_b(a)$	$f(n)$ vs n^p	Case	Result
$T(n) = 4T(n/2) + n$	4, 2	2	$n = O(n^{(2-1)})$	1	$\Theta(n^2)$
$T(n) = 9T(n/3) + n$	9, 3	2	$n = O(n^{(2-1)})$	1	$\Theta(n^2)$
$T(n) = 7T(n/2) + n^2$	7, 2	$\log_2 7 \approx 2.807$	$n^2 = O(n^{(2.807-\epsilon)})$	1	$\Theta(n^{2.807})$
$T(n) = 8T(n/2) + n^2$	8, 2	3	$n^2 = O(n^{(3-1)})$	1	$\Theta(n^3)$
$T(n) = 2T(n/2) + n$	2, 2	1	$n = \Theta(n^{(1)})$	2	$\Theta(n \log n)$
$T(n) = T(2n/3) + 1$	1, 1.5	0	$1 = \Theta(n^{(0)})$	2	$\Theta(\log n)$
$T(n) = T(n/3) + n$	1, 3	0	$n = \Omega(n^{(0+1)})$	3	$\Theta(n)$

15.7 Change of Variable Method

- Used when the recurrence shrinks by $\sqrt{n}$ rather than n/b (Master Theorem doesn't directly apply).
- Substitute $n = 2^m$ (so $m = \log_2 n$), define $S(m) = T(2^m)$, transforming $T(\sqrt{n})$ into $S(m/2)$ — now solvable by Master Theorem, then substitute back $m = \log n$.

Example: $T(n) = T(\sqrt{n}) + 1$
Let $n = 2^m \Rightarrow S(m) = S(m/2) + 1$ [Master: $a=1, b=2, p=0$, $f(m)=1=\Theta(m^0) \Rightarrow$ Case 2 $\Rightarrow S(m)=\Theta(\log m)$]
Since $m = \log n$: $T(n) = \Theta(\log \log n)$

Questions

Q1. [Numerical] What is the complexity of: `for(i=1; i<=n; i=i*3) print(...);`

A. $O(n)$ B. $O(\log n)$ C. $O(\sqrt{n})$ D. $O(n \log n)$

Answer: B. **Why:** i multiplies by a constant (3) each iteration, so the loop runs $O(\log_3 n) = O(\log n)$ times.

Q2. [Numerical] What is the complexity of: `for(i=2; i<=n; i=i*i) print(...);`

A. $O(\log n)$ B. $O(\sqrt{n})$ C. $O(\log \log n)$ D. $O(n)$

Answer: C. **Why:** Squaring i each time means the EXPONENT of i doubles every iteration — this reaches n in $O(\log \log n)$ steps, a classic doubling-exponent pattern.

Q3. [Trap] What is the complexity of: `for(i=1; i<n; i=i+(n/2)) print(...);`

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(\sqrt{n})$

Answer: C. **Why:** The step size $(n/2)$ itself scales with n , so the loop always runs about 2-3 times regardless of n 's value — a constant number of iterations, $O(1)$.

Q4. [Numerical] `rec(n): if(n==1) return 1; else return rec(n-1)+rec(n-1)+n;` — what are the time and space complexity?

A. $O(n)$, $O(n)$ B. $O(2^n)$, $O(n)$ C. $O(2^n)$, $O(2^n)$ D. $O(n^2)$, $O(n)$

Answer: B. **Why:** $T(n)=2T(n-1)+O(1)$ gives $O(2^n)$ time, but since calls execute sequentially (not all simultaneously live), max stack depth is only $O(n)$.

Q5. [Trap] `rec(n): if(n==1) return 1; else return 2*rec(n-1)+n;` — how many recursive calls does this make per invocation, and what is $T(n)$?

A. Two calls; $T(n)=O(2^n)$ B. One call; $T(n)=O(n)$ C. One call; $T(n)=O(n^2)$ D. Two calls; $T(n)=O(n^2)$

Answer: B. **Why:** "2*rec(n-1)" is a single call whose returned value is scaled by 2 — not two separate calls. $T(n)=T(n-1)+O(1)=O(n)$.

Q6. [Numerical] Using the Master Theorem, solve $T(n) = 8T(n/2) + n^2$.

A. $\Theta(n^2)$ B. $\Theta(n^2 \log n)$ C. $\Theta(n^3)$ D. $\Theta(n \log n)$

Answer: C. **Why:** $p = \log_2 8 = 3$; $f(n) = n^2 = O(n^{(3-1)}) \rightarrow$ Case 1 $\rightarrow T(n) = \Theta(n^p) = \Theta(n^3)$.

Q7. [Numerical] Solve $T(n) = T(n/3) + n$ using the Master Theorem.

A. $\Theta(n)$ B. $\Theta(n \log n)$ C. $\Theta(\log n)$ D. $\Theta(n^{\log_3 1})$

Answer: A. **Why:** $p = \log_3 1 = 0$; $f(n) = n = \Omega(n^{(0+1)})$, and the regularity condition holds $\rightarrow$ Case 3 $\rightarrow T(n) = \Theta(f(n)) = \Theta(n)$.

Q8. [Numerical] Solve $T(n) = T(\sqrt{n}) + 1$ using change of variable.

A. $\Theta(\log n)$ B. $\Theta(\log \log n)$ C. $\Theta(\sqrt{n})$ D. $\Theta(1)$

Answer: B. **Why:** Substituting $n = 2^m$ gives $S(m) = S(m/2) + 1 = \Theta(\log m)$ by Master Case 2, and $m = \log n$, so $T(n) = \Theta(\log \log n)$.

Q9. Which asymptotic notation gives a TIGHT bound (both upper and lower)?

A. Big-O B. Big-Omega C. Big-Theta D. little-o

Answer: C. **Why:** Theta sandwiches $f(n)$ between $C_1 \cdot g(n)$ and $C_2 \cdot g(n)$ — both bounds hold, unlike O (upper only) or Omega (lower only).

Q10. [Conceptual] Why is worst-case analysis preferred over average-case analysis in most algorithm design?

A. Worst-case is always faster to compute B. Average-case requires knowing the input probability distribution, which is often unknown/intractable, while worst-case gives a reliable guarantee C. Worst-case and average-case are always identical D. Average-case analysis is not mathematically valid

Answer: B. **Why:** Average-case analysis needs a model of "typical" input distribution, which is rarely known precisely; worst-case guarantees reliability regardless of input, making it the most practically useful and commonly reported metric.

16. Sorting Algorithms

16.1 Key Vocabulary

- **In-place/Internal**: needs no extra memory beyond the input (e.g., Heap Sort). **External**: needs additional memory (e.g., Merge Sort's $O(n)$ buffer).
- **Stable**: preserves the relative order of EQUAL elements. **Unstable**: does not guarantee this.

16.2 Algorithm-by-Algorithm Reference

- **Selection Sort**: repeatedly find the min of the unsorted part, swap it to the front. $O(n^2)$ in **every** case (best=worst=average) — it always scans the full remaining unsorted part regardless of input order. **TRAP**: unlike Bubble/Insertion, Selection Sort has **no best-case speedup** even on already-sorted input. Unstable (naive swap can reorder equal elements). Key advantage: makes at most **$n-1$ swaps total** — the fewest of any comparison sort — useful when swap cost is high.
- **Bubble Sort**: repeatedly swap adjacent out-of-order pairs, "bubbling" larger elements to the end. $O(n^2)$ average/worst. **TRAP**: $O(n)$ best case **only if an early-exit flag** detects a pass with zero swaps (already sorted) — without the flag, it's $O(n^2)$ even on sorted input. Stable.
- **Insertion Sort**: builds the sorted portion by inserting each new element into its correct position via adjacent comparisons+shifts. $O(n)$ best case (already sorted — comparison fails immediately, no shifting), $O(n^2)$ average/worst. Stable. **Online** (can sort a stream as elements arrive) and best for small/nearly-sorted datasets.
- **Merge Sort**: divide-and-conquer — split in half, recursively sort, then **merge**. $O(n \log n)$ in **every** case (best=average=worst — its key selling point over Quick Sort). $O(n)$ **extra** space for the merge buffer (External). Stable. Recurrence: $T(n) = 2T(n/2) + n$.
- **Quick Sort**: divide-and-conquer via pivot + partition (elements < pivot go left, > pivot go right). $O(n \log n)$ average/best, but **$O(n^2)$ worst case** — occurs when the pivot is always the smallest/largest element (e.g., already-sorted input with naive first/last-element pivot choice). **TRAP**: **randomized pivot selection** mitigates this worst case in practice, which is why production Quick Sort implementations randomize. $O(\log n)$ space (recursion stack, average case). **Unstable** (in-place partition swaps break relative order).
- **Heap Sort**: build a Max-Heap, then repeatedly extract the max to the end. $O(n \log n)$ in **every** case. $O(1)$ **extra** space (fully in-place) — **TRAP**: this $O(1)$ -space + guaranteed- $O(n \log n)$ -worst-case combination is exactly why Heap Sort is preferred over Quick Sort when worst-case guarantees matter, and over Merge Sort when memory is constrained. Unstable.

Master Comparison Table

Algorithm	Best	Average	Worst	Extra Space	Stable?
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Bubble	$O(n)^*$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

(Bubble's $O(n)$ best case requires the early-termination flag.)

16.3 The Comparison-Sort Lower Bound

Any comparison-based sorting algorithm needs at least $\Omega(n \log n)$ comparisons in the worst case.

- **Why**: there are $n!$ possible orderings of n elements; each comparison yields at most 1 bit of information (branches a decision tree in 2); so at least $\log_2(n!) = \Omega(n \log n)$ comparisons are needed to distinguish all $n!$ outcomes (decision-tree argument).
- **TRAP**: This bound applies **only to comparison-based sorts**. Non-comparison sorts can beat it under special assumptions: **Counting Sort** — $O(n+k)$ where k = range of input values, stable, needs $O(k)$ extra space; works only for a bounded/small range of integer keys, not general comparable data.

Questions

Q1. [Trap] What is the best-case time complexity of Bubble Sort?

A. $O(n \log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(\log n)$

Answer: B. **Why:** With the early-exit flag optimization, an already-sorted array triggers zero swaps in the first pass, so the algorithm terminates after one $O(n)$ pass.

Q2. [Trap] What is the worst-case time complexity of Insertion Sort when the insertion POSITION is found via binary search instead of linear scan?

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(n(\log n)^2)$

Answer: C. **Why:** Binary search reduces the number of COMPARISONS to $O(\log n)$ per insertion, but the actual SHIFTING of elements to make room is still $O(n)$ per insertion in an array — so the overall worst case remains $O(n^2)$; this is a classic trap (comparisons $\neq$ total work).

Q3. Which of these sorting algorithms has the lowest WORST-CASE time complexity?

A. Quick Sort B. Bubble Sort C. Merge Sort D. Selection Sort

Answer: C. **Why:** Merge Sort guarantees $O(n \log n)$ even in the worst case; Quick Sort degrades to $O(n^2)$, and Bubble/Selection are always $O(n^2)$.

Q4. [Numerical/GATE] After the FIRST partitioning step of Quicksort on an 8-element array, it looks like: 2 5 1 7 9 12 11 10. Which statement is correct about the pivot?

A. The pivot could be either the 7 or the 9 B. The pivot could be the 7, but not the 9 C. The pivot is not the 7, but could be the 9 D. Neither 7 nor 9 is the pivot

Answer: A. **Why:** Checking the partition property (all elements left $\leq$ value $\leq$ all elements right): for 7 — left {2,5,1} all ≤ 7 , right {9,12,11,10} all > 7 $\checkmark$. For 9 — left {2,5,1,7} all ≤ 9 , right {12,11,10} all > 9 $\checkmark$. Both independently satisfy the partitioned property, so either could have been the pivot (classic GATE question).

Q5. What is the worst-case recurrence for Quicksort, and its resulting time complexity?

A. $T(n)=T(n-2)+O(n)$, $O(n^2)$ B. $T(n)=T(n-1)+O(n)$, $O(n^2)$ C. $T(n)=2T(n/2)+O(n)$, $O(n \log n)$ D. $T(n)=T(n/10)+T(9n/10)+O(n)$, $O(n \log n)$

Answer: B. **Why:** Worst case occurs when the pivot always splits the array into sizes 0 and $n-1$ (e.g., already-sorted input with last-element pivot); $T(n)=T(n-1)+O(n)$ solves to $O(n^2)$.

Q6. [Trap] Which sorting algorithm is generally NOT stable in its typical implementation?

A. Insertion Sort B. Merge Sort C. Quick Sort D. Bubble Sort

Answer: C. **Why:** Standard in-place partitioning in Quick Sort can swap equal elements past each other, breaking their original relative order.

Q7. Which algorithm gives the best performance on an array that is already sorted or has just 1-2 misplaced elements?

A. Quick Sort B. Heap Sort C. Merge Sort D. Insertion Sort

Answer: D. Why: Insertion Sort's comparisons fail almost immediately when data is already in order, giving near- $O(n)$ performance — the best adaptation to nearly-sorted input among these options.

Q8. [Trap] If swap operations are very costly, which sorting algorithm should be preferred to minimize the total number of swaps?

A. Heap Sort B. Selection Sort C. Insertion Sort D. Merge Sort

Answer: B. Why: Selection Sort performs at most $n-1$ swaps total (one swap per outer iteration), the fewest of any comparison sort, even though its comparison count is still $O(n^2)$.

Q9. [Numerical] When all elements of the input array are IDENTICAL, which of these (Insertion/Heap/Merge/Selection Sort) takes the LEAST time in a typical implementation?

A. Insertion Sort B. Heap Sort C. Merge Sort D. Selection Sort

Answer: A. Why: With all-equal elements, Insertion Sort's inner-loop condition ($A[i] > \text{key}$) is false immediately for every element — no shifting ever happens, giving $O(n)$ total; Heap/Merge Sort still do $O(n \log n)$ regardless of content, and Selection Sort always does $O(n^2)$ comparisons.

Q10. Why can Counting Sort achieve $O(n+k)$ time, beating the $\Omega(n \log n)$ comparison-sort lower bound?

A. It secretly still compares elements pairwise B. It is not a comparison-based algorithm — it uses key values directly as array indices, so the decision-tree lower bound doesn't apply C. The lower bound is only theoretical and never actually holds D. It only works on already-sorted arrays

Answer: B. Why: The $\Omega(n \log n)$ bound applies specifically to algorithms that sort purely via pairwise comparisons; Counting Sort instead uses the key values directly to place elements, sidestepping the comparison-based decision-tree argument entirely (at the cost of needing a bounded key range k).

Q11. Merge Sort's space complexity is $O(n)$ extra, while Heap Sort's is $O(1)$ extra. Both achieve $O(n \log n)$ worst case. What is the main practical tradeoff?

A. There is no tradeoff — they are identical in every way B. Heap Sort is preferred under memory constraints; Merge Sort is preferred for stability and works better on sequential/external storage C. Merge Sort is always slower D. Heap Sort is always unstable and therefore never used

Answer: B. Why: Merge Sort's stability and sequential-access pattern suit external/linked data; Heap Sort's in-place $O(1)$ space suits memory-constrained internal sorting, at the cost of being unstable.

17. Searching Algorithms & Min-Max

17.1 Linear vs Binary Search

Feature	Linear Search	Binary Search
Precondition	None — works on unsorted data	Requires sorted array
Best case	$O(1)$	$O(1)$
Average/Worst	$O(n)$	$O(\log n)$
Space	$O(1)$	Iterative $O(1)$, Recursive $O(\log n)$ (recursion stack)

TRAP: Binary search's $O(\log n)$ speed is worthless if you forget it **requires sorted input** — sorting an unsorted array just to binary-search it once costs $O(n \log n)$, worse than a single $O(n)$ linear scan. Binary search pays off only when you search the SAME sorted data many times.

Binary Search — mechanism & recurrence

Each comparison eliminates half the remaining search space:

$$T(n) = T(n/2) + O(1) \Rightarrow T(n) = \Theta(\log n)$$

Worked trace — search for 47 in [3,9,15,22,31,40,47,58,66,72] (indices 0-9)

```
low=0 high=9 mid=4 -> A[4]=31 < 47 -> low=5
low=5 high=9 mid=7 -> A[7]=58 > 47 -> high=6
low=5 high=6 mid=5 -> A[5]=40 < 47 -> low=6
low=6 high=6 mid=6 -> A[6]=47 == 47 -> FOUND at index 6 (3 comparisons)
```

- Binary search may be **iterative** ($O(1)$ space) or **recursive** ($O(\log n)$ stack space, since each call needs its own low/high/mid frame).
- Not efficient on a **linked list** (no $O(1)$ random access to the middle element — must walk the list, defeating the purpose).

Recognition Pattern

If a query is issued **once**, plain linear scan (or a single sort+scan) may be cheaper overall. If the SAME dataset is searched **repeatedly**, sort once (amortize the $O(n \log n)$ cost) then binary-search each query in $O(\log n)$ — this trade-off is a common conceptual MCQ.

17.2 Min-Max Algorithm (find both min AND max together)

Straight (linear) method — compare each element to running min AND max

```
Best case (array increasing): n-1 comparisons (each element only beats "max")
Worst case (array decreasing): 2(n-1) comparisons (each element tested against BOTH max and min)
Average case: (3n/2) - 1.5 comparisons
```

Divide & Conquer method — split into halves, recurse, merge (compare max-vs-max, min-vs-min)

```
Recurrence: T(n) = 2T(n/2) + 2
Comparisons (n a power of 2): ceil(3n/2) - 2
```

TRAP: D&C Min-Max needs only **~1.5n** comparisons vs. the straight method's **up to 2n** — a rare case where restructuring comparisons (not the asymptotic complexity, both are $O(n)$) still meaningfully reduces the constant factor, a classic "efficient algorithm" numerical question.

Worked Example

Q. Comparisons needed by an efficient (D&C) Min-Max algorithm for $n = 250$ elements?
 $\text{ceil}(3 \times 250 / 2) - 2 = \text{ceil}(375) - 2 = \mathbf{373}$.

Questions

Q1. Binary search on an array of 1024 sorted elements takes at most how many comparisons in the worst case?

A. 1024 B. 512 C. 10 D. 100

Answer: C. Why: $\log_2(1024) = 10$ — each comparison halves the search space.

Q2. [Trap] Which statement about binary search is FALSE?

A. It requires sorted data B. Its worst-case complexity is $O(\log n)$ C. It is efficient for singly linked lists D. Recursive implementation uses $O(\log n)$ stack space

Answer: C. Why: Binary search needs $O(1)$ random access to the middle element; a linked list only allows $O(n)$ sequential access, defeating binary search's advantage.

Q3. [Numerical] Using the straight Min-Max method on a strictly decreasing array of 20 elements, how many comparisons are performed?

A. 19 B. 20 C. 38 D. 40

Answer: C. Why: Worst case = $2(n-1) = 2(19) = 38$ (every new element beats the running min, forcing both comparisons).

Q4. [Numerical] How many comparisons does the Divide-and-Conquer Min-Max algorithm need for $n = 64$ elements?

A. 63 B. 94 C. 126 D. 96

Answer: B. Why: $\text{ceil}(3 \times 64 / 2) - 2 = 96 - 2 = 94$.

Q5. Why is it inefficient to binary-search an array that will only ever be queried once?

A. Binary search never works on unsorted data and there's no way around it B. Sorting first costs $O(n \log n)$, which dominates the $O(\log n)$ search benefit for a single query — a linear scan ($O(n)$) would be cheaper overall C. Binary search always requires more memory than linear search D. There is no inefficiency; binary search is always preferred

Answer: B. Why: The one-time cost of sorting ($O(n \log n)$) exceeds a single linear scan's $O(n)$, so binary search only pays off when the sort cost is amortized across many repeated queries.

Q6. In the recursive binary search implementation, what causes the recursion to terminate with "not found"?

A. mid equals key B. low exceeds high (search space exhausted) C. The array is empty from the start D. A stack overflow occurs

Answer: B. **Why:** The search space shrinks by half each call; once $\text{low} > \text{high}$, no elements remain to check, so the key is confirmed absent.

18. Greedy Algorithms

18.1 Greedy Paradigm

- Makes the **locally optimal** choice at each step, hoping it leads to a global optimum; decisions are **never revisited** (contrast with DP, which may reconsider via subproblem tables — see Ch.19).
- Works only when the problem has: **Greedy-choice property** (a locally optimal choice leads to a globally optimal solution) + **Optimal substructure** (an optimal solution contains optimal solutions to subproblems).
- **TRAP:** Greedy does **NOT** always yield the optimal answer — classic counterexample: **0/1 Knapsack** (greedy by any single criterion can fail; needs DP — Ch.19), while **Fractional Knapsack** IS correctly solved by greedy (see §18.3).

18.2 Huffman Coding

- Builds an **optimal prefix-free** (no code is a prefix of another) variable-length binary code by repeatedly merging the **two lowest-frequency** nodes into a new node (weight = sum) until one tree remains.
- More frequent symbols get **shorter** codes. Time complexity: **$O(n \log n)$** (using a min-heap: n extract-min + insert operations, each $O(\log n)$).
- Built **bottom-up** (contrast: Shannon-Fano builds top-down and is generally sub-optimal vs Huffman).

Worked Example — 5 symbols, frequencies $M_1=12, M_2=4, M_3=45, M_4=17, M_5=23$ (total=101)

```
Merge steps (always combine two smallest):
M2(4)+M1(12) = 16 [N1]
N1(16)+M4(17) = 33 [N2]
M5(23)+N2(33) = 56 [N3]
M3(45)+N3(56) = 101 [root]

Resulting codes (one valid left/right assignment):
M3 = 0      (1 bit)
M5 = 10     (2 bits)
M4 = 111    (3 bits)
M1 = 1101   (4 bits)
M2 = 1100   (4 bits)

Total bits = 12(4) + 4(4) + 45(1) + 17(3) + 23(2)
           = 48 + 16 + 45 + 51 + 46 = 206 bits
Fixed-length alternative: ceil(log2 5)=3 bits/symbol x 101 = 303 bits
Bits saved = 303 - 206 = 97 bits
```

TRAP: Huffman codes are **not unique** — swapping the 0/1 assignment at any node (or breaking merge ties differently) gives an equally-valid, equally-optimal code with the SAME bit lengths. A question may list two different-looking bit strings for the same word and both can be correct (see Q3 below).

Worked Example 2 — frequencies $a=5, b=9, c=12, d=13, e=16, f=45$ (total=100)

```
Merges: a5+b9=14[N1]; c12+d13=25[N2]; N1(14)+e16=30[N3]; N2(25)+N3(30)=55[N4]; f45+N4(55)=100[root]
Codes:  f=0, c=100, d=101, a=1100, b=1101, e=111
Total Huffman bits = 45(1)+12(3)+13(3)+16(3)+5(4)+9(4) = 45+36+39+48+20+36 = 224
Fixed-length: ceil(log2 6)=3 bits x 100 = 300 bits
Bits saved = 300 - 224 = 76 bits
Code for "dead" = d+e+a+d = 101+111+1100+101 = 1011111100101 (13 bits)
```

18.3 Optimal Merge Pattern

- Problem: merge multiple sorted files into one; merging files of size m and n costs **$m+n$** record movements.
- Greedy rule: always merge the **two smallest** files first — this is exactly Huffman's merge logic applied to file sizes (minimizes total weighted movement).

Worked Example — 8 files, page counts: 18,3,15,12,10,11,7,9

```
Sorted: 3,7,9,10,11,12,15,18
Merge 3+7=10 (running total 10)
Merge 9+10=19 (total 29)
Merge 10+11=21 (total 50)
Merge 12+15=27 (total 77)
Merge 18+19=37 (total 114)
Merge 21+27=48 (total 162)
Merge 37+48=85 (total 247)
Minimum total record movements = 247
```

18.4 Knapsack — Fractional (Greedy WORKS) vs 0/1 (Greedy FAILS)

- **Fractional Knapsack:** items are divisible. Sort by **value/weight ratio descending**, take as much of the best-ratio item as fits, then move to the next. This greedy strategy is **provably optimal**.
- **0/1 Knapsack:** items are indivisible (all-or-nothing). Greedy by ratio (or profit, or weight alone) can be **sub-optimal** — must use Dynamic Programming (Ch.19).
- **TRAP:** Greedy by **profit alone** or **weight alone** is WRONG even for the fractional version — only the **ratio (profit/weight)** ordering guarantees optimality.

Worked Example — capacity $W=20$; $O_1(\text{profit}25, \text{weight}18)$, $O_2(\text{profit}24, \text{weight}15)$, $O_3(\text{profit}15, \text{weight}10)$

Ratios: $O1=1.39$, $O2=1.60$, $O3=1.50$ → optimal order: $O2$, $O3$, $O1$

Greedy by RATIO (optimal): take $O2$ fully ($w15, p24$), capacity left=5.
Take $5/10$ of $O3$: profit += $15 \times (5/10) = 7.5$. TOTAL = 31.5 <- OPTIMAL

Greedy by PROFIT alone (wrong order): $O1$ first ($w18, p25$), capacity left=2.
Take $2/15$ of $O2$: profit += $24 \times (2/15) = 3.2$. TOTAL = 28.2 (sub-optimal)

Greedy by WEIGHT alone (wrong order): $O3$ first ($w10, p15$), capacity left=10.
Take $10/15$ of $O2$: profit += $24 \times (10/15) = 16$. TOTAL = 31 (sub-optimal, close but not best)

18.5 Job Sequencing with Deadlines

- n jobs, each with a deadline and profit; each job takes 1 time unit; **one job per time slot**; maximize total profit.
- Algorithm: (1) Sort jobs by profit **descending**. (2) For each job, place it in the **latest free slot** at or before its deadline. (3) If no such slot exists, discard the job.

Worked Example — $T1(P=100, D=2)$, $T2(P=10, D=1)$, $T3(P=27, D=2)$, $T4(P=15, D=1)$

Sort by profit: $T1(100, d2)$, $T3(27, d2)$, $T4(15, d1)$, $T2(10, d1)$
 $T1(d2)$: slot 2 free → assign. Profit=100
 $T3(d2)$: slot 2 taken → try slot 1 → free → assign. Profit=127
 $T4(d1)$: slot 1 taken → no earlier slot → REJECT
 $T2(d1)$: slot 1 taken → REJECT
 Maximum profit = 127 (jobs $T1$, $T3$ scheduled)

Questions

Q1. [Trap] For which problem does a straightforward greedy strategy generally FAIL to find the optimal solution?

A. Fractional Knapsack B. Huffman Coding C. 0/1 Knapsack D. Optimal Merge Pattern

Answer: C. **Why:** 0/1 Knapsack's indivisibility breaks the greedy-choice property; only Dynamic Programming guarantees the optimal solution here.

Q2. [Numerical] Using the frequencies $M1=12$, $M2=4$, $M3=45$, $M4=17$, $M5=23$, what is the total number of bits needed to Huffman-encode a message with these character counts?

A. 303 B. 206 C. 224 D. 101

Answer: B. **Why:** Sum of (frequency × code length): $12(4)+4(4)+45(1)+17(3)+23(2) = 206$ bits (see worked example above).

Q3. [Trap] For the frequency set $a=5, b=9, c=12, d=13, e=16, f=45$, the word "dead" can be validly encoded as EITHER 101111100101 OR its bit-complement 0100000011010. Why are both correct?

A. Only one can be correct; the other is an error B. The 0/1 labeling of left/right children at each merge is an arbitrary convention — swapping it everywhere yields an equally valid, equally optimal code with identical lengths C. Huffman coding is non-deterministic and gives random results D. Because "dead" has repeated letters

Answer: B. **Why:** Huffman optimality depends only on code LENGTHS (determined by tree structure/depth), not which branch is labeled 0 vs 1 — both labelings are valid, prefix-free, and equally optimal.

Q4. What is the time complexity of building a Huffman code for n symbols using a min-heap?

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(n (\log n)^2)$

Answer: B. **Why:** $n-1$ merge steps, each requiring $O(\log n)$ heap extract-min and insert operations.

Q5. [Numerical] 8 files of sizes 18, 3, 15, 12, 10, 11, 7, 9 pages are merged two at a time (optimal merge pattern). What is the minimum total number of record movements?

A. 162 B. 200 C. 247 D. 300

Answer: C. **Why:** Repeatedly merging the two smallest values (Huffman-style) gives cumulative cost $10+19+21+27+37+48+85 = 247$.

Q6. [Numerical] Knapsack capacity=20; items $O1(\text{profit}25, w18)$, $O2(\text{profit}24, w15)$, $O3(\text{profit}15, w10)$. Using the CORRECT (ratio-based) greedy strategy for Fractional Knapsack, what is the maximum profit?

A. 28.2 B. 31 C. 31.5 D. 34

Answer: C. **Why:** Sort by ratio ($O2=1.6$, $O3=1.5$, $O1=1.39$): take $O2$ fully (profit 24), then $5/10$ of $O3$ (profit 7.5) = 31.5 total — the true optimum (profit-only or weight-only ordering give 28.2 or 31, both sub-optimal).

Q7. [Numerical] Jobs $T1(\text{profit}100, \text{deadline}2)$, $T2(\text{profit}10, \text{deadline}1)$, $T3(\text{profit}27, \text{deadline}2)$, $T4(\text{profit}15, \text{deadline}1)$ — single machine, 1 unit per job. What is the maximum achievable profit?

A. 100 B. 115 C. 127 D. 152

Answer: C. **Why:** Scheduling $T1$ in slot 2 and $T3$ in slot 1 (both meet their deadlines) yields $100+27=127$; $T2$ and $T4$ cannot be scheduled since both slots by their deadline (1) are already used.

Q8. Why does the Fractional Knapsack greedy strategy require sorting by value/weight RATIO rather than by value or weight alone?

A. Ratio sorting is just a convention with no mathematical necessity B. Only ratio ordering guarantees the maximum value gained per unit of capacity consumed at every step, which is what optimality requires C. Weight-based sorting is always equivalent to ratio-based sorting D. Profit-based sorting always gives the same result as ratio-based sorting

Answer: B. **Why:** The greedy-choice property for Fractional Knapsack specifically requires maximizing value-per-unit-weight at each greedy step; sorting by profit or weight alone ignores this ratio and can pick a worse combination (as shown in the worked example: 28.2 or 31 vs the true optimum 31.5).

19. Dynamic Programming

19.1 DP Paradigm

- Like Divide & Conquer, DP combines solutions to subproblems — but DP applies when subproblems are **NOT independent**, i.e., subproblems **overlap** (share subproblems). Each distinct subproblem is solved **once** and its answer is **cached/tabulated**, avoiding recomputation.
- TRAP:** This overlapping-subproblems property is exactly what **distinguishes DP from plain Divide & Conquer** — Merge Sort's subproblems never overlap, so memoization gives it no benefit; DP problems (LCS, MCM, Knapsack) have massively overlapping subproblems, so memoization gives an exponential-to-polynomial speedup.
- Two implementation styles: **Top-down/Memoization** (recursive + cache — same asymptotic complexity, but risks stack depth for large n) vs **Bottom-up/Tabulation** (iterative, build the table from the smallest subproblems up — no recursion overhead).
- 4 standard steps: (1) Characterize the structure of an optimal solution. (2) Recursively define its value. (3) Compute it bottom-up. (4) Reconstruct the actual solution from the table.

19.2 Longest Common Subsequence (LCS)

```
C[i,j] = 0                      if i=0 or j=0
C[i,j] = C[i-1,j-1] + 1        if x_i = y_j
C[i,j] = max( C[i-1,j], C[i,j-1] )  if x_i != y_j
```

- Time & space: **O(m x n)** for the table; reconstructing the actual LCS string takes an additional O(m+n).

Worked Example — X = "ABCBADAB" (m=7), Y = "BDCABA" (n=6)

Filling the C[i,j] table per the recurrence above gives **LCS length = 4**, e.g. the subsequence **"BCBA"** (or equivalently "BDAB") — both are valid longest common subsequences of the same length.

19.3 Matrix Chain Multiplication (MCM)

- Goal: choose the **order of multiplication** (parenthesization) of a chain of matrices to minimize total scalar multiplications — the matrices themselves are NOT actually multiplied, only the optimal split point is found.
- If matrix A_i has dimensions P_(i-1) x P_(i):

```
m[i,i] = 0
m[i,j] = min over k in [i,j-1] of ( m[i,k] + m[k+1,j] + P(i-1)*P(k)*P(j) )
```

- Time complexity: **O(n^3)** (O(n^2) subproblems, O(n) choices of k each).

Worked Example 1 — A₁(30x35), A₂(35x15), A₃(15x5), A₄(5x10), A₅(10x20)

Minimum scalar multiplications = **15125** (classic result; optimal split groups as ((A₁(A₂ A₃))(A₄ A₅))).

Worked Example 2 — M₁(10x100), M₂(100x20), M₃(20x5), M₄(5x80) [P = 10,100,20,5,80]

```
m[1,2]=10*100*20=20000      m[2,3]=100*20*5=10000      m[3,4]=20*5*80=8000
m[1,3]=min(k=1: 0+10000+10*100*5=15000, k=2: 20000+0+10*20*5=21000) = 15000
m[2,4]=min(k=2: 0+8000+100*20*80=168000, k=3: 10000+0+100*5*80=50000) = 50000
m[1,4]=min(
  k=1: 0+50000+10*100*80=130000,
  k=2: 20000+8000+10*20*80=44000,
  k=3: 15000+0+10*5*80=19000    <- MINIMUM
)
Minimum scalar multiplications = 19000
(optimal grouping: M1 * ((M2*M3)*M4) )
```

TRAP: Two "obvious" parenthesizations of this same chain give 44000 and 25000 multiplications — NEITHER is optimal. The true minimum (19000) requires checking **all** split points via the DP recurrence, not just eyeballing a couple of groupings.

19.4 Floyd-Warshall (All-Pairs Shortest Path)

```
dist[i][j] = min( dist[i][j], dist[i][k] + dist[k][j] )  for each intermediate vertex k = 1..n
```

- O(n^3) time; works with **negative edge weights** but NOT with a **negative-weight cycle** (result becomes meaningless/undefined along that cycle).

Worked Example — 3 vertices, direct edges: 1->2=5, 2->3=2, 3->1=1 (others = infinity)

Initial:	After k=1:	After k=2:	After k=3 (final):
1 2 3	1 2 3	1 2 3	1 2 3
1 0 5 INF	1 0 5 INF	1 0 5 7	1 0 5 7
2 INF 0 2	2 INF 0 2	2 INF 0 2	2 3 0 2
3 1 INF 0	3 1 6 0	3 1 6 0	3 1 6 0

Final shortest distances: d(1,3)=7 (via 1->2->3), d(2,1)=3 (via 2->3->1), d(3,2)=6 (via 3->1->2)

19.5 Subset Sum Problem

```
dp[i][s] = dp[i-1][s] OR dp[i-1][s - arr[i]]  (True if a subset of first i elements sums to s)
```

Worked Example — S = {2, 3, 7, 8, 10}, target sum = 14

Checking every one of the 31 non-empty subsets systematically (singles, pairs, triples, quadruples, and the full set) — **no subset sums to exactly 14** (closest are 13 and 15). **Answer: No such subset exists.**

TRAP: Don't assume a subset always exists just because the target is "in range" ($2 \leq 14 \leq 30 = \text{total sum}$) — the DP table must be checked exactly; existence is not guaranteed.

Questions

Q1. [Trap] What is the key property that makes Dynamic Programming applicable but NOT plain Divide & Conquer?

- A. The problem must be recursive B. Subproblems must overlap (shared sub-subproblems), unlike D&C's independent subproblems C. The problem must involve sorting D. DP only works on graphs

Answer: B. Why: DP's advantage (caching) only pays off when subproblems recur; D&C problems like Merge Sort have non-overlapping subproblems, so there's nothing to cache.

Q2. [Numerical] For $X = \text{"ABCBADAB"}$ and $Y = \text{"BDCABA"}$, what is the length of the Longest Common Subsequence?

- A. 3 B. 4 C. 5 D. 6

Answer: B. Why: Applying the LCS recurrence yields length 4 (e.g., "BCBA").

Q3. [Numerical] Matrices $A_1(30 \times 35)$, $A_2(35 \times 15)$, $A_3(15 \times 5)$, $A_4(5 \times 10)$, $A_5(10 \times 20)$ — minimum scalar multiplications to compute $A_1A_2A_3A_4A_5$?

- A. 30000 B. 15125 C. 7875 D. 45500

Answer: B. Why: This is the classic MCM DP result — 15125, achieved via the specific split $((A_1(A_2A_3))(A_4A_5))$.

Q4. [Numerical/Trap] For $M_1(10 \times 100)$, $M_2(100 \times 20)$, $M_3(20 \times 5)$, $M_4(5 \times 80)$, which grouping gives the TRUE minimum multiplication count?

- A. $((M_1M_2)(M_3M_4))$ with 44000 B. $((((M_1M_2)M_3)M_4))$ with 25000 C. $(M_1((M_2M_3)M_4))$ with 19000 D. All groupings give the same cost

Answer: C. Why: Full DP evaluation of all split points shows grouping as $M_1((M_2M_3)M_4)$ costs only 19000 — lower than either of the two "obvious" groupings (44000, 25000).

Q5. What is the time complexity of the Matrix Chain Multiplication DP algorithm for n matrices?

- A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(n^3)$

Answer: D. Why: $O(n^2)$ subproblems (i, j pairs), each requiring an $O(n)$ scan over split points k .

Q6. Floyd-Warshall's all-pairs shortest path algorithm fails to give meaningful results when the graph contains:

- A. Negative edge weights B. A negative-weight cycle C. Disconnected components D. Self-loops with weight 0

Answer: B. Why: Floyd-Warshall correctly handles individual negative edges, but a negative cycle allows unbounded decreasing path weight, making shortest-path distances undefined along it.

Q7. [Numerical/Trap] Does the set $S = \{2, 3, 7, 8, 10\}$ contain a subset summing to exactly 14?

- A. Yes, $\{2, 3, \dots\}$ sums to 14 B. Yes, $\{7, \dots\}$ sums to 14 C. No such subset exists D. Cannot be determined without more information

Answer: C. Why: Systematically checking all 31 non-empty subsets shows none sum to exactly 14 (13 and 15 are the closest achievable sums).

Q8. Top-down (memoized) and Bottom-up (tabulated) DP implementations differ mainly in:

- A. Asymptotic time complexity — tabulation is always faster B. Tabulation avoids recursion-call/stack overhead by building the table iteratively from smallest subproblems, while memoization uses recursion with caching C. Only memoization gives the correct answer D. Tabulation cannot reconstruct the actual optimal solution, only its value

Answer: B. Why: Both achieve the same asymptotic complexity; tabulation's iterative bottom-up build avoids the function-call and stack-depth overhead that recursive memoization carries.

20. Graph Algorithms — MST & Shortest Path

20.1 Minimum Spanning Tree (MST)

- A **spanning tree** of a connected graph G is a subgraph that is a tree AND includes every vertex of G . An MST is the spanning tree with **minimum total edge weight**.
- A graph with n vertices has an MST with **exactly $n-1$ edges** (this fact is heavily tested).
- MST is defined only for **connected** graphs; a disconnected graph instead has a **minimum spanning forest** (union of each component's MST).
- Multiple MSTs can exist if edge weights tie, but the **total MST weight is always unique**.

20.2 Kruskal's Algorithm (edge-centric, Greedy)

```

1. Sort all edges by weight, ascending.
2. For each edge (u,v) in that order:
    if Find_Set(u) != Find_Set(v):    # doesn't create a cycle
        add edge to MST; Union(u,v)
3. Stop once n-1 edges are added.
```

- Cycle detection via **Union-Find (Disjoint Set)**. Time: **$O(E \log E)$** (dominated by the sort).
- Best suited for **sparse** graphs (edge-list based).

20.3 Prim's Algorithm (vertex-centric, Greedy)

```

1. Start from any vertex; tree = {start}.
2. Repeatedly add the MINIMUM weight edge connecting the tree to a vertex OUTSIDE the tree.
3. Stop when all vertices are included.
```

- Time: **$O(E \log V)$** with a min-heap (or $O(V^2)$ with a simple array — better for **dense** graphs).
- **TRAP:** Kruskal's picks edges **globally** by weight (needs Union-Find to avoid cycles); Prim's grows a **single connected tree** greedily from a chosen source — different mechanisms, but both are provably correct and give the **same total MST weight**.

Worked Example — vertices A,B,C,D,E; edges A-B=2, A-C=3, B-C=1, B-D=4, C-D=5, C-E=6, D-E=2

```

KRUSKAL (sort: BC=1, AB=2, DE=2, AC=3, BD=4, CD=5, CE=6):
BC(1): add      {B,C}
AB(2): add      {A,B,C}
DE(2): add      {D,E}
AC(3): SKIP (A,C already same component -> would cycle)
BD(4): add -> connects {A,B,C} and {D,E}    ALL CONNECTED (4 edges)
Total weight = 1+2+2+4 = 9
```

```

PRIM (start at A):
Add B via A-B=2      (tree wt=2)
Add C via B-C=1      (tree wt=3)
Add D via B-D=4      (tree wt=7)
Add E via D-E=2      (tree wt=9)
Total weight = 9    <- MATCHES Kruskal's total
```

20.4 Dijkstra's Algorithm (Single-Source Shortest Path)

```

1. dist[source]=0, all others=infinity.
2. Repeatedly extract the UNVISITED vertex with smallest known dist; mark it visited (finalized).
3. RELAX every neighbor v of u: if dist[v] > dist[u] + w(u,v): dist[v] = dist[u] + w(u,v)
```

- Time: **$O(E \log V)$** with a min-heap. Requires **non-negative** edge weights.
- **TRAP:** Dijkstra **fails with negative edge weights** — once a vertex is "finalized" (visited), its distance is assumed final, but a later negative edge could still improve it. Use **Bellman-Ford** for graphs with negative weights.

Worked Example — Dijkstra from source A on the same graph as §20.3

```

Visit A(0): relax B->2, C->3
Visit B(2): relax D-> 2+4=6      (C via B: 2+1=3, no improvement, tie)
Visit C(3): relax E-> 3+6=9      (D via C: 3+5=8, no improvement, 6 already better)
Visit D(6): relax E-> 6+2=8      (improves 9 -> 8)
Visit E(8): done

Final shortest distances from A:  A=0, B=2, C=3, D=6, E=8
```

20.5 Bellman-Ford Algorithm

```

1. dist[source]=0, all others=infinity.
2. Relax ALL edges, repeated (V-1) times.
3. One more pass: if any edge can STILL be relaxed -> a negative-weight cycle exists (report it).
```

- Time: **$O(V \times E)$** — slower than Dijkstra, but works with **negative edges** and can **detect negative cycles** (a capability Dijkstra has no equivalent for at all).

20.6 Comparison Table

Algorithm	Purpose	Handles negative weights?	Complexity
Kruskal	MST	N/A (undirected weights, can be negative too — still works)	$O(E \log E)$
Prim	MST	N/A	$O(E \log V)$
Dijkstra	Single-source shortest path	No	$O(E \log V)$
Bellman-Ford	Single-source shortest path	Yes , and detects negative cycles	$O(V \cdot E)$

20.7 Classic Conceptual Traps (verified True/False)

- **"If all edge weights in a graph are unique (no ties), the shortest path between any two vertices is always unique."** → **TRUE**. Distinct weights eliminate the possibility of two different paths tying for minimum total weight.
- **"With all-positive weights, Dijkstra and Bellman-Ford may report different actual PATHS, but the path WEIGHT is always the same."** → **TRUE**. Both algorithms correctly compute the shortest-path *weight*; only the specific path chosen among ties can differ.
- **"If every edge weight is doubled, the shortest path (which edges are used) stays the same — only the total weight doubles."** → **TRUE**. Multiplying all weights by a positive constant preserves the relative ordering of all path lengths.
- **TRAP — "If every edge weight is increased by the SAME constant (e.g., +1), the shortest path always remains the same."** → **FALSE**. Adding a constant per edge disproportionately penalizes paths with MORE edges (hops). Counter-example: direct edge $A \rightarrow B = 10$ (1 edge) vs. a 3-edge path totaling 9 (3 edges of weight 3 each) — the 3-edge path is shorter ($9 < 10$). Add 1 to every edge: direct becomes 11 (1 edge), the 3-edge path becomes $3 \times 4 = 12$ (3 edges) — now the DIRECT edge is shorter ($11 < 12$). The shortest path **flipped**.
- **"Which of these is NOT a Greedy algorithm: Dijkstra / Prim / Kruskal / Huffman / Bellman-Ford?"** → **Bellman-Ford** (it systematically relaxes all edges $V-1$ times rather than making a locally-greedy choice; it's relaxation/DP-flavored, not greedy).
- **"Which of these is NOT Dynamic-Programming based: Bellman-Ford / Floyd-Warshall / 0-1 Knapsack / Prim's MST?"** → **Prim's MST** (it's Greedy; the other three are classic DP algorithms).

Questions

Q1. A connected graph has 12 vertices. How many edges does its Minimum Spanning Tree contain?

A. 12 B. 11 C. 13 D. Depends on the graph's total edges

Answer: B. Why: An MST always has exactly $(n-1)$ edges for n vertices — here $12-1=11$.

Q2. [Numerical] Using the graph $A-B=2$, $A-C=3$, $B-C=1$, $B-D=4$, $C-D=5$, $C-E=6$, $D-E=2$, what is the total MST weight?

A. 7 B. 9 C. 11 D. 13

Answer: B. Why: Both Kruskal's (edges BC, AB, DE, BD) and Prim's independently arrive at total weight $1+2+2+4=9$.

Q3. [Trap] Dijkstra's algorithm is unsuitable for a graph with:

A. Many vertices B. Cycles C. Negative edge weights D. Disconnected components

Answer: C. Why: Dijkstra assumes a finalized vertex's distance can never improve — a later negative edge can violate this assumption, producing incorrect results.

Q4. [Numerical] Using Dijkstra from source A on $A-B=2$, $A-C=3$, $B-C=1$, $B-D=4$, $C-D=5$, $C-E=6$, $D-E=2$, what is the shortest distance from A to E?

A. 6 B. 8 C. 9 D. 11

Answer: B. Why: Shortest path $A \rightarrow B \rightarrow D \rightarrow E = 2+4+2 = 8$, beating $A \rightarrow C \rightarrow E$ ($3+6=9$).

Q5. Which algorithm can both compute shortest paths with negative edges AND detect a negative-weight cycle?

A. Dijkstra's B. Prim's C. Kruskal's D. Bellman-Ford

Answer: D. Why: Bellman-Ford's extra (V -th) relaxation pass reveals a negative cycle if any edge can still be relaxed — a capability unique to it among these four.

Q6. [Trap] "If every edge's weight is increased by the same constant, the shortest path never changes." This statement is:

A. True — additive constants never affect relative path comparisons B. False — paths with more edges are penalized more than paths with fewer edges, which can flip which path is shortest C. True only for undirected graphs D. True only when the constant is negative

Answer: B. Why: A constant added per edge scales with the number of edges in each path, so a longer (more-hop) path that was originally shorter can become longer than a shorter-hop path after the addition (see the worked 1-edge-vs-3-edge counter-example above).

Q7. Which of the following is NOT a Greedy algorithm?

A. Prim's MST B. Kruskal's MST C. Huffman Coding D. Bellman-Ford Shortest Path

Answer: D. Why: Bellman-Ford performs systematic edge relaxation over $V-1$ rounds rather than a locally greedy selection; the other three make a clear greedy choice at each step.

Q8. Which of the following is NOT Dynamic-Programming based?

A. Bellman-Ford B. Floyd-Warshall C. 0-1 Knapsack D. Prim's MST

Answer: D. Why: Prim's is a Greedy algorithm; the other three rely on DP recurrences over subproblems.

Q9. Why is Kruskal's algorithm generally preferred for SPARSE graphs, while Prim's (array-based) is preferred for DENSE graphs?

A. Kruskal's cannot run on dense graphs at all B. Kruskal's cost is dominated by sorting E edges (efficient when E is small relative to V^2), while a simple Prim's implementation scans all vertices each step regardless of edge count, which pays off only when E is large C. Prim's cannot handle sparse graphs D. There is no such distinction; both always perform identically

Answer: B. Why: Kruskal's $O(E \log E)$ scales with edge count, favoring sparse graphs; a simple $O(V^2)$ Prim's implementation performs consistently regardless of E , so it becomes relatively more efficient as graphs get denser (E close to V^2).

21. String Matching Algorithms

21.1 The Problem

- Find all occurrences of a **Pattern (P, length m)** within a **Text (T, length n)**.
- 4 standard approaches: Naive, Rabin-Karp, Finite Automata-based, and Knuth-Morris-Pratt (KMP).

21.2 Naive (Brute-Force) String Matching

```
for i = 1 to (n-m+1):
    compare P[1..m] against T[i..i+m-1] character by character
    if all match: report a hit at position i
```

- Tries **every** possible alignment ($n-m+1$ positions), comparing character by character.
- Complexity: **Best case $O(n)$** (mismatches found immediately at most positions), **Average/Worst case $O(n-m)$** (e.g., text of all A's, pattern of all A's-then-B — many near-full matches before the final mismatch). Space: **$O(1)$** .

Worked Example — $T = \text{"AABAACAADAABAABA"} (n=16)$, $P = \text{"AABA"} (m=4)$

Checking every alignment position by position: **matches found at positions 1, 10, and 13** (1-based indexing).

21.3 Rabin-Karp Algorithm

```
1. Compute hash(P).
2. Compute hash of the first length-m window of T.
3. If hash(window) == hash(P): verify with a full character comparison (rule out hash collision).
4. Slide the window by 1; update its hash in  $O(1)$  via a ROLLING HASH (remove outgoing char's contribution, shift, add incoming char) — NOT recompute
5. Repeat until the text is exhausted.
```

- Hash Collision:** two different substrings can hash to the same value — character verification is mandatory whenever hashes match, to rule out a false positive.
- Complexity: **Best/Average case $O(n+m)$** (rolling hash update is $O(1)$ per shift, verification is rare); **Worst case $O(n-m)$** if hash collisions are frequent (e.g., a poorly chosen hash function). Space: **$O(1)$** .
- Advantages: faster than Naive on average; efficient when searching for **multiple patterns simultaneously** (precompute multiple pattern hashes, scan text once).
- Disadvantages: performance degrades with hash collisions; needs a good hash function.

Worked Example — $P = \text{"31415"} (m=5)$, $T = \text{"2359023141526739921"} (n=21)$

Rolling-hash comparison finds the hash match (then verifies) at **position 7** of T : $T[7..11] = \text{"31415"}$ — an exact match.

21.4 Knuth-Morris-Pratt (KMP) — brief contrast

- Precomputes a **failure function / prefix table** (longest proper prefix of P that is also a suffix, for every prefix of P) so that on a mismatch, the pattern is shifted intelligently — **never re-examining a text character already matched**.
- Achieves a guaranteed **$O(n+m)$ worst case** — unlike Rabin-Karp, which can degrade to $O(n-m)$ under hash collisions, and unlike Naive's $O(n-m)$ worst case. [VERIFY: exact prefix-table construction steps if a numeric KMP trace question appears — the source material for this chapter did not cover KMP's mechanics in depth, only its existence as a standard companion algorithm.]

21.5 Summary Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Key Idea
Naive	$O(n)$	$O(n-m)$	$O(n-m)$	$O(1)$	Brute-force compare at every position
Rabin-Karp	$O(n+m)$	$O(n+m)$	$O(n-m)$	$O(1)$	Rolling hash, verify on hash match
KMP	$O(n+m)$	$O(n+m)$	$O(n+m)$ guaranteed	$O(m)$ (prefix table)	Precomputed failure function, no re-scanning

TRAP: Rabin-Karp's *average-case* speed is the same order as KMP's *guaranteed worst case* — but Rabin-Karp can still degrade to $O(n-m)$ under adversarial input with many hash collisions, while KMP **never** does. When a worst-case guarantee matters, KMP is the stronger choice despite similar typical-case performance.

Questions

Q1. [Numerical] For $T = \text{"AABAACAADAABAABA"}$ and $P = \text{"AABA"}$, at which positions (1-indexed) does the Naive algorithm report a match?

A. 1, 5, 9 B. 1, 10, 13 C. 1, 4, 10 D. 10, 13, 16

Answer: B. **Why:** Direct character-by-character comparison at each alignment confirms exact matches only at positions 1, 10, and 13.

Q2. What is the worst-case time complexity of the Naive string matching algorithm?

A. $O(n+m)$ B. $O(n \log m)$ C. $O(n-m)$ D. $O(\log n)$

Answer: C. **Why:** In the worst case (e.g., highly repetitive text), nearly every alignment requires close to m comparisons before a mismatch, across $n-m+1$ positions.

Q3. In Rabin-Karp, why is a character-by-character comparison still needed even after a hash match is found?

A. It's not needed; hash match alone confirms a true match B. To rule out a hash collision, where two different substrings share the same hash value C. To recompute the hash from scratch D. Only needed if the pattern length exceeds 10

Answer: B. **Why:** A hash match is necessary but not sufficient for a true match — distinct strings can collide to the same hash value, so verification is required.

Q4. [Numerical] Searching for pattern "31415" in text "2359023141526739921" using Rabin-Karp, at what position does the true match occur?

A. Position 3 B. Position 5 C. Position 7 D. No match exists

Answer: C. **Why:** $T[7..11] = \text{"31415"}$, an exact character match confirming the hash hit at position 7.

Q5. [Trap] Which string matching algorithm guarantees $O(n+m)$ even in the WORST case, unlike Rabin-Karp which can degrade under hash collisions?

A. Naive B. Rabin-Karp C. KMP D. None can guarantee this

Answer: C. **Why:** KMP's precomputed failure function ensures text characters are never re-examined, giving a true worst-case $O(n+m)$ bound that Rabin-Karp cannot guarantee under adversarial hash collisions.

Q6. What technique allows Rabin-Karp to update its window hash in $O(1)$ time as it slides across the text?

A. Recomputing the hash from scratch each time B. Rolling Hash (remove the outgoing character's contribution, shift, add the incoming character) C. Sorting the characters in the window D. Using a separate thread for each window

Answer: B. **Why:** The rolling hash technique incrementally updates the hash value in constant time instead of rehashing the entire window, which is what gives Rabin-Karp its average-case speed advantage.